Computer Science Review 57 (2025) 100749

Contents lists available at ScienceDirect

Computer Science Review

journal homepage: www.elsevier.com/locate/cosrev

Traffic load balancing in data center networks: A comprehensive survey ☆

Guisheng Liu [a], Yong Liu [a,*], Qian Meng [b,*], Ben Wang [a], Kefei Chen [b], Zhonghua Shen [b]

[a] School of Information Science and Technology, Hangzhou Normal University, Hangzhou, 311121, Zhejiang, China
[b] School of Mathematics, Hangzhou Normal University, Hangzhou, 311121, Zhejiang, China

ARTICLE INFO

Keywords:
Data center networks
Load balancing
Machine learning
Traffic optimization

ABSTRACT

The rapid growth in scale and complexity of data centers has established effective load balancing as a critical requirement for optimizing network performance, resource utilization, and service quality. This survey provides a comprehensive examination of load balancing strategies in data center networks, addressing challenges, established techniques, and emerging trends. The study begins with fundamental concepts of data center networks, encompassing network topologies, load balancing definitions, granularity considerations, and inherent challenges. A systematic analysis of load balancing schemes follows, categorized into centralized and distributed approaches. Within centralized schemes, the analysis encompasses both machine learning-based approaches and traditional methodologies, while distributed schemes are examined through single-granularity and mixed-granularity implementations. The survey extends beyond previous reviews by offering comparative analyses of load balancing strategies, a systematic evaluation of performance metrics, and an assessment of their applicability across diverse data center scenarios. The discussion further explores future development trends driven by technological innovations across multiple dimensions, including hardware, software, architecture, and algorithms. This comprehensive technical review serves as a foundational resource for researchers and practitioners, facilitating advancements in efficient, scalable, and user-centric load balancing technologies for data center networks.

Contents

1. Introduction 2
2. Background 3
   2.1. Data center network topologies 4
   2.2. Granularity in data center networks 5
   2.3. Definition of load balancing 6
   2.4. Challenges in load balancing 6
   2.5. Scenarios and industrial practice of load balancing 7
      2.5.1. Key application scenarios of load balancing in production 7
      2.5.2. Industrial practice of load balancing 8
      2.5.3. Session-aware load balancing 10
3. Centralized schemes of load balancing in data center networks 10
   3.1. ML-based schemes 11
      3.1.1. Supervised learning schemes 12
      3.1.2. Reinforcement learning 12
   3.2. Conventional schemes 14
      3.2.1. Energy-aware schemes 14
      3.2.2. Energy-agnostic schemes 15
   3.3. Discussion of centralized schemes 16
4. Distributed schemes 17

☆ This work was supported by the National Natural Science Foundation of China under Grant 62402154 and Grant 62302136, the Zhejiang Provincial Natural Science Foundation of China under Grant No. LQ22F010001, and the Starting Research Funds from the Hangzhou Normal University under Grant 4115C50221204137 and 4085C50220204093.

* Corresponding authors: Yong Liu, Qian Meng.
E-mail addresses: yongliu@hznu.edu.cn (Y. Liu), mq@hznu.edu.cn (Q. Meng).

https://doi.org/10.1016/j.cosrev.2025.100749
Received 14 October 2024; Received in revised form 23 February 2025; Accepted 16 March 2025
Available online 28 March 2025
1574-0137/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

4.1. Single-granularity schemes 17
4.1.1. Flow-based schemes 17
4.1.2. Flowlet-based schemes 19
4.1.3. Flowcell-based schemes 20
4.1.4. Packet-based schemes 20
4.2. Mixed-granularity 21
4.2.1. Multi-granularity schemes 21
4.2.2. Adaptive-granularity schemes 21
4.3. Discussion of distributed schemes 22
5. Summary and future insights 23
5.1. Summary of metrics in load balancing 23
5.2. Quantitative analysis of centralized and distributed schemes 25
5.3. Future insights 26
5.3.1. Load balancing innovation based on emerging network hardware 26
5.3.2. Efficient load balancing based on user-space protocol stacks and NFV 26
5.3.3. Load balancing architecture design for multi-point coordination 26
5.3.4. Deep integration of machine learning and load balancing strategies 26
5.3.5. Open issues: Energy and large language models 26
6. Conclusion 27
Declaration of competing interest 27
Data availability 27
References 27

1. Introduction

With the rapid development of the Internet, data center networks have become a key infrastructure to support the modern information society. According to Cisco's report [1], global data center traffic will reach 20.6 ZB in 2023, which is 3.2 times that of 2018. With the increasing size of data centers and the growing complexity of application requirements, meeting these demands has become a key challenge for data centers. Data centers not only need to handle massive data traffic, but also support the computing power requirements of cutting-edge technologies like cloud computing, big data analytics and artificial intelligence. Currently, many companies such as Huawei, Google, Amazon, etc. are specifically for their own needs to provide the corresponding reliable services. Effective load balancing strategies are necessary to ensure network performance, improve resource utilization and ensure quality of service.

The core function of load balancing in data center networks is to reasonably distribute network traffic and computing tasks across multiple links or servers while ensuring network condition constraints are met. This process not only optimizes resource utilization, but also contributes to reducing network congestion, enhancing system reliability and improving user experience. However, implementing effective load balancing faces a number of challenges, including handling heterogeneous traffic, resolving network topology asymmetries, and ensuring low latency while maintaining high throughput.

To address these issues, early load balancing schemes in data center networks, such as ECMP [2] routing and hash-based techniques, provided basic functionality for traffic distribution. However, when faced with dynamic network environments and complex traffic patterns, these approaches revealed significant limitations, including poor adaptability, coarse granularity, and lack of global view. Established schemes have also tried hard on these issues: e.g., CONGA [3] for globally sensing congestion, DRILL [4], which is based on random selection, Hermes [5] for sensing real-time network state, and Presto [6] based on flow segmentation. In response to the growing scale and complexity of network environments, researchers have proposed various load balancing algorithms from different perspectives in recent years [7–9].

Although these methods have improved load balancing, new challenges continue to emerge as data center networks continue to scale up and application requirements rapidly evolve. These new challenges include not only traditional load balancing issues, but also heterogeneous computing demands, low latency guarantees, and more. Meanwhile, the introduction of new technologies such as artificial intelligence has brought new design ideas and implementation approaches to load balancing. These new developments and trends urgently need a systematic summary and analysis to help researchers and practitioners better understand the advantages and limitations of existing schemes, and to provide guidance for future research directions. Furthermore, load balancing schemes face many challenges in moving from theoretical research to practical deployment, requiring in-depth discussion of specific application scenarios and implementation considerations. This motivated us to conduct this survey study. To help readers better understand this paper, A comprehensive list of technical terms and abbreviations is provided in Table 1.

Table 1
List of abbreviations.

Abbreviation	Full name
ToR	Top of Rack
EoR	End of Row
MCF	Multi-Commodity Flow
MF	Mice Flow
EF	Elephant Flow
IoT	Internet of Things
MEC	Mobile Edge Computing
LLM	Large Language Model
RPC	Remote Procedure Call
PCC	Per Connection Consistency
RDMA	Remote Direct Memory Access
ANN	Artificial Neural Network
NN	Neural Network
KDN	Knowledge-Defined Networking
NIC	Network Interface Card
DDPG	Deep Deterministic Policy Gradient
CNN	Convolutional Neural Network
FCT	Flow Completion Time
TD3	Twin Delayed Deep Deterministic Policy Gradient
PPO	Proximal Policy Optimization
ILP	Integer Linear Programming
INT	In-band Network Telemetry
RLNC	Random Linear Network Coding

Given the extensive research on data center load balancing, a comprehensive review of recent survey papers in this field was conducted to establish the context of the current work. The comparative analysis of existing surveys evaluates multiple dimensions: publication year, literature coverage period, load balancing classifications, machine learning algorithm consideration, future research directions, practical implementation scenarios, and quantitative experimental analyses. The comparative results are presented in Table 2.

Table 2
Comparison of recent load balancing surveys in data centers.

Ref	Year	Coverage	Type	ML-Related	Insights	Real scenarios	Experimental analysis
Zhang et al. [10]	2018	2010–2018	Link LB	✗	✓	✗	✗
Thakur et al. [11]	2017	2009–2016	Server LB	✗	✓	✗	✗
Ghomi et al. [12]	2017	2009–2017	Server LB	✗	✓	✓	✗
Hasan et al. [13]	2022	2010–2016	Server LB	✗	✓	✗	✗
Muchori et al. [14]	2022	2018–2021	Server LB	✓	✓	✗	✗
Li et al. [15]	2021	2010–2020	Link LB	✗	✗	✗	✗
Hacham et al. [16]	2022	2016–2021	Both	✗	✓	✗	✗
Wen et al. [17]	2019	2014–2017	Link LB	✗	✗	✗	✓
Wang et al. [18]	2022	2018–2021	Link LB	✓	✓	✗	✗
Du et al. [19]	2024	2020–2023	Link LB	✗	✓	✗	✗
Liu et al. [20]	2024	2020–2023	Link LB	✓	✓	✗	✓
Ours	–	2018–2024	Link LB	✓	✓	✓	✓

Looking at the timeline, surveys on load balancing are relatively evenly distributed. The closest work to ours is Zhang et al. [10], which reviewed most load balancing schemes from the 2010s to 2018, provided basic preliminary knowledge, classified schemes based on power awareness and congestion information usage, and discussed key points in design schemes. However, it did not involve quantitative analysis. In [17], the authors presented their experimental results, which was also addressed in the [20]. In [12], adopted a scenario-based classification approach, providing readers with a clearer view. Furthermore, with the development of artificial intelligence technology, considering the application of machine learning algorithms in load balancing has become a crucial perspective.

This work builds upon the strengths of previous surveys while extending their scope and depth. The classification framework incorporates recent developments in machine learning algorithms. Notably, this study addresses a significant gap in existing literature by providing comprehensive coverage of real-world applications and practical deployment schemes in data centers and data center networks. The background section presents detailed analyses of load balancing scenarios, industrial practices of major companies, and commercial load balancing schemes offered by cloud service providers, offering valuable insights for understanding practical implementations. The analysis encompasses recent load balancing schemes, future development trends, and quantitative experimental comparisons. Furthermore, the study examines the architectural distinctions between centralized and distributed approaches, providing clear comparative insights. The key contributions of this work are as follows:

- This study presents a comprehensive survey of recent load balancing schemes in data center networks. Through systematic analysis and synthesis of these approaches, the work establishes a clear technical framework.
- The study presents a comprehensive background framework encompassing data center network topologies, load balancing definitions, granularity considerations, common challenges, and real-world implementations. This foundational analysis establishes the context for subsequent detailed examination of data center load balancing technologies.
- The study establishes distinct evaluation criteria for different categories of load balancing approaches. This targeted framework enables meaningful comparisons across schemes, providing clear insights into the relative strengths and limitations of various approaches within their respective categories.
- The study presents both quantitative comparisons between centralized and distributed approaches and comprehensive insights into design principles and future developments. These contributions enhance the theoretical understanding of load balancing mechanisms and offer direction for future research and implementation.

This study is structured as follows: Section 2 introduces the basic concepts and theoretical models of load balancing in data center networks; Section 3 analyzes in detail the existing centralized load balancing schemes and their strengths and weaknesses; Section 4 analyzes in detail the existing distributed load balancing schemes and their strengths and weaknesses; Section 5 provides a summary of metrics in load balancing, quantitative analysis and some insights about future development; and, finally, Section 6 concludes the full paper. For more detailed content, please refer to Fig. 1.

2. Background

With the continuous development of cloud computing and big data applications, the scale and complexity of data center networks have increased significantly. Therefore, it is necessary to carry out continuous research and in-depth analysis of data center networks and load balancing technologies to highlight their growing importance. As the basic infrastructure supporting cloud computing and big data processing, data center networks are responsible for of transmitting and processing massive data. The performance and efficiency of these networks have a significant impact on the overall system architecture.

Load balancing is a key technology in the field of traffic engineering that optimizes network performance in data center environments by distributing traffic fairly across multiple links. Therefore, a comprehensive study of the fundamentals, methods, and optimization strategies of data center networks and load balancing is essential to improve the performance and stability of these critical systems.

To provide context for our subsequent discussion on load balancing schemes in data center networks, this section focuses on presenting the background related to data center networks and load balancing. These include common data center network topologies, granularity considerations, definition of load balancing and some of the challenges that exist in data center networks. This section addresses the following key components:

- Data center network topologies: A concise review and background description of several typical topologies.
- Granularity in data center networks: An analysis of the granularity commonly used in data center network environments.
- Definition of load balancing: A systematic and comprehensive definition of load balancing using formulas.
- Challenges in load balancing: A comprehensive summary and critical discussion of typical problems encountered in load balancing in data center networks.
- Scenarios and industrial practice of load balancing: A review of application scenarios and industrial practice of load balancing.

The examination of these aspects provides foundational insights into the underlying motivations and theoretical framework of data center network load balancing schemes, establishing a basis for subsequent detailed analysis.



Fig. 1. Road map of the survey.

2.1. Data center network topologies

Data center network architectures vary widely, depending on their operational environments and required functionalities. Among these topologies, the Fat-tree [21] architecture has emerged as the predominant structure, due to its capacity to provide symmetrical bandwidth for both ingress and egress links. The Fat-tree topology is a ubiquitous data center network architecture that facilitates large-scale parallel communication paradigms, offering high bandwidth capacity and enhanced fault tolerance. This structure comprises multiple hierarchical layers of switches: edge switches, aggregation switches, and core switches. The edge layer, situated at the lowest tier, interfaces directly with a number of terminal server ports; the core layer occupies the topmost tier, where all core switches are fully interconnected; the aggregation layer is positioned intermediately between the edge and core layers, with each switch establishing connections to a subset of edge switches and core switches (see Fig. 2).

One of the fundamental design principles of the Fat-tree topology is path redundancy. Within this structure, multiple equivalent shortest paths exist between any two nodes, enabling automatic rerouting to alternative equivalent paths in the event of link failures, thus ensuring high availability. Moreover, Fat-tree exhibits excellent scalability, making it suitable for a wide spectrum of data center environments.

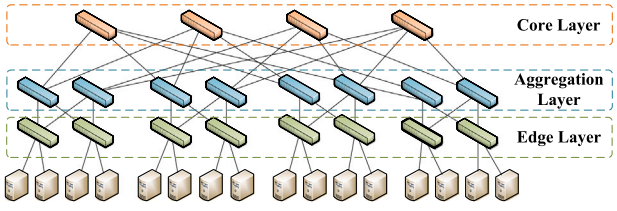


Fig. 2. Fat-tree topology.

With the development of data center networks, an important phenomenon is the gradual shift of the once predominantly north–south traffic within data centers towards a pattern dominated by east–west traffic, which reflects the fact that interactions within data centers are becoming more and more complex with the proliferation of users and evolution of scenarios, while traditional three-tier topologies such as Fat-tree, due to the lack of dedicated paths for east–west traffic, have begun to show some limitations in this scenario. In contrast, two-tier architectures are better able to address these emerging challenges: they require relatively fewer devices and more direct communication paths between switches as the network becomes very large.

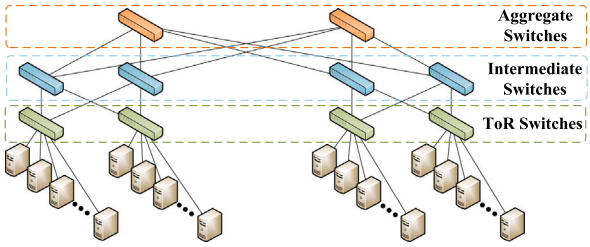


Fig. 3. VL2 topology.

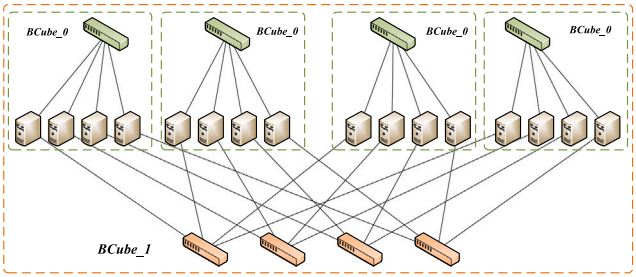


Fig. 4. $BCube_1$ topology.

As a result, the traditional three-layer Clos architecture is gradually being replaced by a two-layer Clos architecture, particularly the Leaf-Spine topology, due to the growing demand for scalability in data centers. This transition has significantly improved network scalability while reducing capital costs and path delays. Among the Leaf-Spine architectures, Microsoft's VL2 [22] is particularly notable. It consists of three switch layers: Top-of-Rack (ToR) switches, intermediate switches, and aggregate switches. All aggregate switches are fully interconnected, forming the core of the network. The intermediate switches are responsible for forwarding traffic between the ToR switches and Aggregate switches, while the ToR switches connect directly to the servers in each rack. It is important to note that the failure of any individual switch only reduces the bidirectional bandwidth by $1/n$, where n represents the number of aggregate switches. This design increases the number of end-to-end paths while ensuring the robustness and high availability of the network (see Fig. 3).

In addition to VL2, Microsoft researchers have proposed a hybrid topology called BCube [23]. This architecture uniquely employs multi-port switches and servers as forwarding devices (see Fig. 4). An intriguing aspect of BCube is its recursive definition, which is different from what we discussed above. BCube_0 is the basic unit of this topology, which consists of n servers interconnected with an n-port switch. BCube_1 consists of n BCube_0 units and n n-port switches. Thus, the recursive expression for the BCube architecture is:

$$\text{BCube}_k = n \times \text{BCube}_{k\text{-}1} + n^k \text{ switches}$$

Recursive definitions in network topologies are not uncommon. DCell [24] is another data center network topology defined recursively. DCell represents a server-centric topology where servers, analogous to switches, possess data forwarding capabilities (see Fig. 5). Its basic unit, termed DCell_0, comprises n servers and one mini-switch, with each server connected to the mini-switch via a single link. A key characteristic of the DCell topology is the absence of direct connections between switches; switches are solely responsible for intra-unit communication. DCell_1 is composed of $n+1$ DCell_0 units. Thus, the recursive expression for DCell_k is:

$$\text{DCell}_k = (n+1) \times \text{DCell}_{k\text{-}1}$$

Beyond the four aforementioned topologies, numerous other architectural designs have emerged in recent years, including Portland [25], Jellyfish [26], RibsNet [27], Lascada [28], Flecube [29], FiConn [30], SDCCP [31], Helios [32], Criso [33], MDCube [34], and HS-DCell [35]. These architectures aim to enhance performance over existing topologies when handling complex traffic patterns. Although these topologies are not commonly utilized as simulation environments for design and performance evaluation of specific schemes, understanding their principles remains valuable for comprehending load balancing in data center networks. While this overview is brief, comprehensive details are available in the relevant literature and reviews.

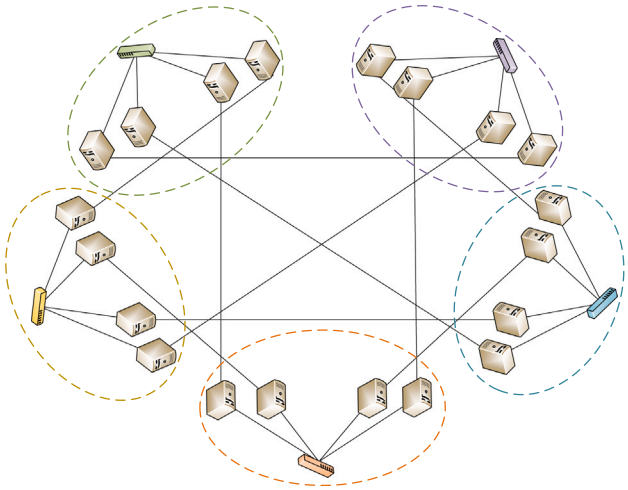


Fig. 5. $DCell_1$ topology.

2.2. Granularity in data center networks

The selection of granularity is crucial in load balancing strategies. These granularities range from flow-level to packet-level, and can be employed based on specific requirements. Different forwarding granularities offer distinct advantages, particularly when facing the highly complex and dynamic environment of data center networks (see Fig. 6).

Flow-level granularity:

Initially, flow-level granularity was adopted in data center load balancing. A flow is defined as a set of packets sharing the same five-tuple identifier. Traditional flow-based load balancing schemes [36–39], route all packets of a flow along a single path using flow-based hashing, thereby avoiding packet reordering. However, flow-based load balancing cannot fully exploit the network's multi-path capabilities, especially in networks dominated by large flows, potentially leading to congestion.

Flowlet-level granularity:

To address the limitations of flow-level granularity, the concept of flowlet was introduced. A flowlet is defined as a sequence of consecutive packets within a flow where the inter-packet gap exceeds a certain time threshold. By introducing this time interval, a flow can be naturally segmented into multiple flowlets, which can be scheduled across different paths. Approaches using flowlet [40–42] significantly improve link utilization. Moreover, as packets within each flowlet remain contiguous, the issue of packet reordering is minimized.

Flowcell-level granularity:

As network speeds continue to increase, flowlet segmentation has become increasingly fine-grained, presenting new challenges for load balancing due to the proliferation of small flowlets. Flowcell involves splitting a flow into fixed-size units, typically around 64 KB. Each flowcell can independently select a path for forwarding. Compared to flowlets, flowcells offer improved stability and predictability, mitigating the increasing risk of packet reordering and network scheduling overhead associated with finer flowlet segmentation [6,43,44].

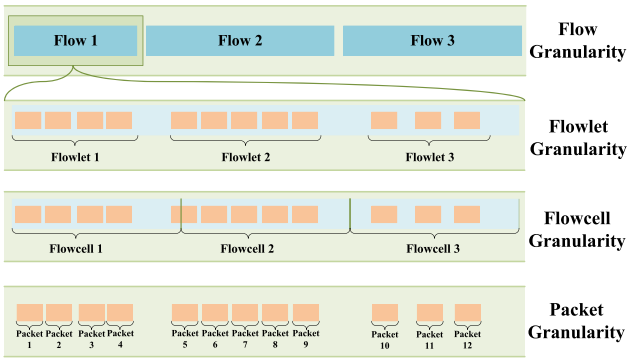


Fig. 6. Different granularity in load balancing.

Packet-level granularity:

Packet-level granularity represents the finest-grained approach in load balancing strategies, offering maximum flexibility in traffic distribution. Packet-level granularity schemes [45–47] allows for more flexible distribution of packets across multiple paths, thereby improving network resource utilization and data transmission efficiency. However, inconsistencies in packet arrival order may result in additional delays and complex reordering operations, which can impact overall network performance.

2.3. Definition of load balancing

In contemporary data center networks, there are multiple equal-cost paths between hosts, and different types of traffic need to be transmitted at the same time. Therefore, load balancing becomes a key technique to ensure the performance of data center networks. Traffic load balancing in a data center network refers to the reasonable and even distribution and scheduling of network traffic to different links to avoid certain links from becoming congested while others are underutilized, thus maximizing the resource utilization and throughput of the whole system and minimizing latency.

Following the introduction to the fundamental concepts and significance of load balancing, a more precise definition is warranted. Traffic scheduling in capacity-constrained networks is typically characterized as a multi-commodity flow problem. The formal definition is presented as follows:

Given a network $G(V, E)$, where V represents the set of nodes and E represents the set of links. For any link $(u, v) \in E$, there is a property $c(u, v)$ denoting the link capacity. There are k flows to be transmitted in the network, and each flow can be defined as $F_k(s, t, d)$, where s represents the start position, t represents the end point, and d represents the demand.

For a flow k passing through link (u, v), it can be defined as $f_k(u, v)$, where the link utilization ℓ is

$$\ell(u, v) = \frac{\sum_{k=1}^{K} f_k(u, v)}{c(u, v)}.$$

The multi-commodity flow problem needs to follow the following restrictions:

Capacity constraint: the sum of flows along edge (u, v) cannot exceed the link capacity of that edge.

$$\sum_{k=1}^{K} f_k(u, v) \le c(u, v), \quad \forall (u, v) \in E.$$

Flow conservation: the flow remains constant during transmission from the origin to the destination.

$$\sum_{q \in V} f_i(u, q) = \sum_{w \in V} f_i(w, u), \quad u \ne s_i, t_i.$$

Demand satisfaction: the sum of traffic from the origin to any reachable point in the network is the same as the initial demand, ensuring complete transmission.

$$\sum_{w \in V} f_i(s_i, w) = d_i \iff \sum_{w \in V} f_i(w, t_i) = d_i.$$

On this basis, the goal of load balancing is to distribute the traffic to different paths as rationally as possible. In a fixed-rate symmetric network, load balancing aims to reduce the gap between the highest and lowest link utilization in the network. We can define it as:

$$\min_{(u,v) \in E} \left(\max_{(u,v) \in E} \ell(u, v) - \min_{(u,v) \in E} \ell(u, v) \right).$$

This is equivalent to the notion of minimizing the maximum link utilization as proposed in previous research [36].

$$\min \max_{(u,v) \in E} \frac{\sum_{s,d} f_{u,v}^{s,d}}{c_{u,v}}.$$

The Multi-Commodity Flow (MCF) problem, being NP-hard, precludes direct optimization for optimal load balancing. To address this computational challenge, researchers have proposed various load balancing approaches. These approaches are examined in detail in the subsequent section.

2.4. Challenges in load balancing

Traffic variations in data center networks are often unpredictable and highly complex. The typical traffic characteristics of data center networks include heterogeneity, burstiness, and volatility. These characteristics are closely related to network topology and significantly impact the performance and efficiency of load balancing in data center networks. This section will focus on the nature of these issues, their potential challenges, and their impacts to better understand and address load balancing problems in data center networks.

Mice-Elephant flow conflict: Conflicts between heterogeneous flows are one of the most common problems in data center networks due to the diverse requirements of network traffic [48]. There are two typical traffic types are mice flows and elephant flows, Mice flows are latency sensitive, while elephant flows require higher available bandwidth. Since Elephant flows are larger and require more bandwidth, their transmission times are typically longer. When a mice flow and an elephant flow are routed on the same link, if the elephant flow is queued before the mice flow, the mice flow behind it may not be able to transmit because the elephant flow in front of it is taking up the bandwidth, thus resulting in severe congestion. This situation can cause the mice flow to time out and fail to meet the low latency requirement. This can severely impact the flow completion time, which is unacceptable in scenarios that primarily involve the transmission of mice flows [49] (see Fig. 7).

Hash collision: Hash collision is another important problem in data center networks, which occurs when multiple elephant flows are routed to the same link [2]. ECMP uses a hash function to determine the path of each flow, which theoretically allows for a balanced distribution of traffic among multiple available links. However, due to potential problems with the design of hash functions, multiple elephant flows may be routed onto the same link, causing severe congestion while other available links are underutilized. This situation is known as hash conflict. Hash collisions not only affect network performance, but can also lead to packet loss and increased latency. Especially in large-scale data centers, a small probability of hash conflicts may also have a significant impact on the overall network performance. To mitigate the hash conflict problem, researchers have proposed a variety of improvement schemes, including dynamic flow scheduling, adaptive load balancing, and other techniques. These schemes aim to allocate network resources more intelligently and reduce the occurrence of hash collisions, thereby improving the overall performance and reliability of data center networks [45].

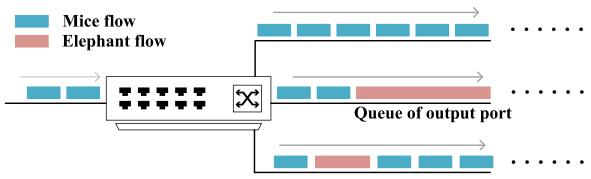


Fig. 7. Flow conflict in data center networks.

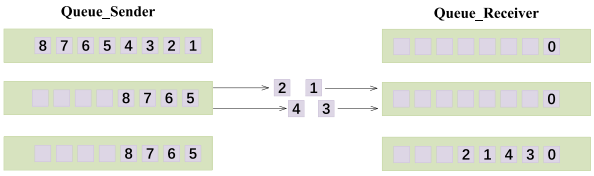


Fig. 8. Packet disordering in data center networks.

Micro-burst flows: Microburst flows are data flows that transmit a small amount of data in a very short period of time but exhibit burstiness. These flows are often triggered by an instantaneous increase in application demand, changes in network topology, or other factors, and are characterized by suddenness, transient nature, and high bandwidth consumption that is difficult to detect [50]. Although the duration is usually in the millisecond to second range, their bursty transmissions are so large that they may cause load pressure on the network within a short period of time, triggering short-term congestion, queue backlogs, and leading to increased packet loss rates and delays. Due to the unpredictable and ephemeral nature of micro-burst flows, it is often difficult for existing monitoring schemes to capture detailed information about these bursts. To this end, BurstRadar [51] proposes a real-time monitoring tool that efficiently detects and characterizes microburst flows in the data plane to address the challenges in network management. In addition, BurstBalancer [52] proposes a new approach to optimize resource allocation for the load balancing problem of microburst flows, aiming to reduce network congestion and improve the efficiency of traffic management in data centers.

Asymmetric topology: As network scales expand and data volumes increase, a network topology with uniform bidirectional bandwidth between all links may become asymmetric due to equipment configuration issues or physical problems leading to bandwidth reduction or single-point failures [53]. Asymmetric topologies can result from link failures causing bandwidth reduction or single-point failures. In asymmetric topologies, the network link bandwidth between certain nodes is imbalanced, potentially leading to overload risks and increased latency on lower-bandwidth links. Single-point failures rendering certain links unavailable can increase the load on available links, risking severe link congestion and consequently affecting overall network performance [54]. To address these challenges, researchers have proposed various approaches, including adaptive routing algorithms, dynamic load balancing techniques, topology-aware traffic engineering, and so on. However, as data centers continue to evolve, the asymmetric topology problem remains an important challenge.

Reordering problem: In the process of data packet transmission across networks, issues such as network topology and routing strategies can lead to discrepancies between the order of packet arrival and the order of sending, affecting data accuracy and integrity [55]. Reordering problems are common in data center networks, particularly due to the high connectivity of current data center network topologies. As network topological complexity increases and transmission granularity decreases, the reordering problem becomes more pronounced (see Fig. 8). When encountering packet disorder, the typical design approach is to request retransmission of all data from the out-of-order packet onwards. This significantly impacts flow completion time and causes redundant data transmission, undoubtedly exacerbating network load imbalance. To mitigate the reordering problem, researchers have proposed various approaches, including improved transport protocols, intelligent routing algorithms, and efficient buffer management techniques [56–58].

2.5. Scenarios and industrial practice of load balancing

This section examines key application scenarios of data centers in production environments and daily operations, including edge computing, GPU cluster training, and high-performance database deployment. These diverse applications, all dependent on data center infrastructure, are implemented through various approaches. The analysis encompasses production load balancing schemes deployed by major technology companies such as Google, ByteDance, Microsoft, Alibaba Cloud, and Huawei, as well as commercial cloud vendors' load balancing services. This comprehensive examination provides insights into both the practical applications of data centers and state-of-the-art deployment methodologies.

2.5.1. Key application scenarios of load balancing in production

Edge Computing and IoT Applications: In the rapid development of IoT, edge computing has become an indispensable technology. By pushing computing resources and data processing capabilities to the network edge, it provides IoT devices with low-latency, high-reliability, and enhanced security support. Particularly in scenarios requiring real-time processing, traditional architectures struggle to meet IoT devices' urgent demands for low latency and real-time processing.

Edge computing emerged to address these challenges. By deploying computing resources at the network edge, it can provide services to IoT devices in close proximity, fundamentally changing the data processing workflow. Under this distributed architecture, large amounts of data can be processed and filtered at edge nodes, with only necessary data transmitted to the cloud. This not only significantly reduces service latency but also effectively relieves backbone network pressure. Meanwhile, the distributed nature of edge computing enhances system reliability, maintaining basic service processing capabilities even when cloud connections are disrupted. Additionally, sensitive data can be processed locally without cloud upload, better meeting data localization and privacy protection requirements. In practice, edge computing has demonstrated immense value across multiple IoT scenarios. In [59], Alqahtani et al. implemented a reliable scheduling and load balancing scheme for request information in edge computing, while Gowri et al. in [60] developed a secure load balancing scheme for edge computing. As IoT technology continues to evolve, edge computing will provide increasingly efficient, secure, and reliable support for IoT applications.

Healthcare Datacenter: Healthcare data centers represent another crucial scenario, distinguished primarily by their highly sensitive data and stringent privacy requirements. With the development of wearable devices and medical IoT, large amounts of user health data are continuously collected and stored in medical data centers. One of the main challenges for healthcare data centers is privacy. While this data involves personal privacy requiring strict access control and protection mechanisms, medical data analysis and utilization are vital for improving public healthcare quality. Therefore, effectively utilizing medical data while protecting privacy has become a key concern for healthcare data centers.

To address these challenges, healthcare data centers typically deploy privacy protection mechanisms. These mechanisms often involve cryptographic techniques, such as homomorphic encryption, enabling data computation and analysis in encrypted states, achieving effective data utilization while protecting privacy. In [61], Zheng et al. proposed a blockchain-based VeriDC protocol, implementing verifiable data classification through blockchain while ensuring data privacy. In [62], Tuli et al. introduced HealthFog, adopting a cloud–edge collaborative architecture that integrates IoT devices and edge computing resources

to efficiently manage cardiac patients' data, reducing data leak risks of traditional cloud-based health systems while providing better service response performance, which is significant for patients. As medical IoT and wearable devices become more prevalent, healthcare data centers will continue to face dual challenges of scale growth and privacy protection. How to efficiently build health monitoring systems and strengthen data privacy based on healthcare data centers remains a major challenge.

Blockchain for Data Centers: Blockchain technology in data centers primarily focuses on trusted distributed storage and processing capabilities. However, besides performance bottlenecks, blockchain systems face complex security challenges, mainly arising from their sequential workflow in transaction execution, consensus building, and result verification, which causes significant delays due to serialization. Bidl [63] parallelizes consensus and execution phases through network ordering capabilities, using dedicated sequencer nodes to assign sequence numbers to transactions and multicast them to all nodes, thereby reducing latency and improving throughput. It employs denial lists to detect and exclude malicious participants, ensuring system security even with malicious nodes present, and allows nodes to execute transactions before consensus based on sequence numbers, ensuring consistency and handling non-deterministic transactions to reduce transaction termination. In future research, besides ensuring basic security, research directions for data center blockchain include introducing cryptographic tools like zero-knowledge proofs and secure multi-party computation, offering new approaches to privacy protection in blockchain systems. Performance optimization remains crucial; in review [64], the authors detailed load balancing issues in blockchain networks, referenced here for interested readers to explore further.

High Frequency Trading in Data Center: Real-time trading represents a key scenario. Currently, millions of transactions occur every second, and any latency can potentially cause substantial losses. In financial exchanges, designing data center networks for high-frequency trading is a topic of strong practical value. As a crucial application scenario for data centers, high-frequency trading's most notable characteristic is its extreme sensitivity to latency. In modern financial markets, microsecond-level latency differences can result in missed trading opportunities, imposing extremely stringent requirements on data center network architecture and system design. Traditional data center architectures face numerous challenges in high-frequency trading: unpredictable delays caused by network congestion, performance bottlenecks at the hardware architecture level, and coordination overhead inherent in distributed systems, all potentially affecting trading fairness and efficiency. At the data center network level, researchers introduced logical clocks and delivery clocks [65], focusing on resolving latency differences between market participants in real-time trading, achieving fair trading without complex clock synchronization mechanisms, ensuring considerable fairness. In hardware architecture, data centers continue evolving to meet high-frequency trading needs. Traditional RDMA and Ethernet struggle to provide ideal performance due to complex protocol stacks and resource management mechanisms, while new PCIe non-transparent bridge architecture opens new paths for data center internal communication, significantly reducing system latency, undoubtedly providing guarantees for latency-sensitive scenarios like real-time trading. As data centers evolve towards serverless architecture, [66] addresses how to achieve ultra-low latency for financial applications, proposing optimizations across multiple levels including functions, platforms, architecture, and infrastructure, demonstrating the feasibility of achieving millisecond-level latency in serverless architecture.

Large-scale GPU Cluster Training: With the continuous expansion of deep learning models, especially the rise of large language models (LLMs), large-scale GPU cluster training has become a crucial application scenario for data centers, as major companies have establishing their own clusters of thousands of GPUs. In such scenarios, data centers need to coordinate thousands of GPUs for efficient collaboration, posing severe challenges to computational resource scheduling, network architecture, and training frameworks. Due to the high cost of GPUs, the hardware specifications within a single data center can be quite complex. On this foundation, effectively utilizing heterogeneous GPU resources has become a critical challenge. Sia [67] schedules adaptive learning tasks on heterogeneous resources to optimize cluster performance. Additionally, network communication resources are also a crucial component. When applying deep learning training in large-scale data centers, network communication often becomes a bottleneck. Cassini [68] optimizes resource allocation and reduces communication bottlenecks in large-scale training tasks through network-aware scheduling algorithms, improving training efficiency. The importance of GPU cluster data centers will continue to grow in the future, remaining a promising application scenario.

Distributed Database Datacenter: With the popularization of cloud computing and explosive data growth, supporting distributed databases has emerged as a primary use case for data centers. Distributed databases enhance storage capacity and processing power by distributing data across multiple nodes, effectively handling large-scale data processing requests. Distributed databases require high reliability, scalability and other key requirements, all closely tied to data center capabilities. Additionally, supporting geographically distributed applications is another key feature of distributed databases. Currently, many enterprises have global operations requiring data sharing and processing across different regions. Through off-site backup and distributed storage, improved performance and enhanced reliability can be achieved. Distributed databases will continue to rely on data centers for deployment, with data centers providing resource pooling, high-speed networks, elastic scaling, and other capabilities providing the fundamental infrastructure for distributed databases. The growing demands of distributed databases will continue to drive progress in data center resource scheduling, network architecture, and other aspects.

2.5.2. Industrial practice of load balancing

This section examines the load balancing deployment strategies of major companies. Due to their complex business requirements, traffic load balancing typically constitutes only one component of their comprehensive solutions. For thoroughness, the complete architectures are presented, with particular emphasis on the load balancing components.

The scenarios above demonstrate different applications of load balancing in data centers. In practice, many large companies have implemented load balancing algorithms according to their specific data center characteristics and published their findings. We will discuss these industry practices in load balancing.

Different vendors implement load balancing strategies according to their distinct business objectives. As business complexity increases, the adopted load balancing schemes operate across multiple levels, ranging from network-layer link load balancing to application-layer load balancer implementations. Despite numerous academic publications proposing novel load balancing schemes in recent years, these organizations maintain a conservative approach in production environments, particularly regarding lower-layer implementations. This prudent stance reflects their commitment to business stability and operational reliability. For instance, Google's Maglev [69] continues to utilize ECMP for link-level load balancing, exemplifying this conservative approach towards lower-layer schemes.

In recent years, major companies have not stopped exploring. Lower-layer schemes have progressed, while higher-layer schemes have flourished. In 2019, Google proposed Tars [70], a higher-layer load balancing scheme. Targeting their core search engine business, it addressed the problem of similar keywords not being focused on the same server by implementing a cache-aware server load balancing system. Under traditional load balancing, simple random query allocation would result in similar but different keywords being handled by different replica servers, leading to low cache hit rates. Tars analyzes terms and selects replicas with the highest cache hit rates online to

save flash bandwidth. Production deployment showed Tars successfully reduced flash I/O, increased CPU utilization, and improved overall throughput.

However, they found that in production server load balancing, serious errors could occur even when some service replicas had not reached their CPU utilization limits, due to other tasks running on the same machines. In 2024, they proposed Prequal [71], a probe-based server load balancing scheme that selects servers based on collected server states through probes. In YouTube's production deployment, it effectively improved latency and resource utilization.

This demonstrates Google's approach on server load balancing. They also worked on link load balancing schemes, reflected in PLB [72]. PLB can operate independently or combine with other schemes. The motivation came from observing that even when overall network capacity was available, some links would frequently congest while others remained underutilized. PLB's main approach avoids major modifications to switches and protocol stacks, instead detecting congestion through transport layer protocols at host endpoints and using IPv6's Flow Label field for path switching. When congestion is detected, it modifies this field value, prioritizing switches during idle connection periods to minimize packet reordering impact. PLB is deployed in Google's data centers, achieving QoS improvements and better load balancing quality. After enabling PLB, both small and large RPC transmission latency performance improved compared to non-PLB schemes. As shown in Fig. 9, mixed PLB deployment (2:8) showed reduced latency compared to no PLB deployment, with greater reduction compared to the baseline.

As discussed previously, large-scale GPU deep learning training represents a crucial scenario in modern data centers. A key challenge is achieving efficient load balancing in heterogeneous GPU cluster data centers. In modern data centers running deep learning training tasks, GPU resources often have very high utilization rates, while CPU resource utilization remains low, and network bandwidth is often underutilized. BytePS [73] addresses this by separating gradient aggregation from parameter updates, preventing CPU from becoming a performance bottleneck, while deploying topology-aware communication optimization with different strategies for different topologies. BytePS is deployed in ByteDance's internal GPU data centers, significantly outperforming existing schemes.

This heterogeneity brings new challenges: due to different GPU configurations, performance gaps between tasks on different generations of GPUs can be significant. To address these issues, Gandivafair [74] was proposed, implementing task migration through hierarchical scheduling to maintain load balancing, along with a core automated resource trading mechanism to balance workloads across heterogeneous resources.

Load balancing practices in data centers for distributed databases is another area frequently addressed by major companies. ByteDance has shared their thoughts on this scenario through ByteGraph [75], their distributed graph database. ByteGraph adopts a dual-layer architecture separating computation and storage layers, allowing the system to scale resources independently based on different types of workloads. ByteGraph implements load balancing through two strategies: dynamic thread pool management and weighted consistent hashing. In real production, ByteGraph maintains low error rates and very stable response times, keeping p99 latency within 60 ms.

These examples illustrate load balancing implementations in production data centers operated by major companies. Contemporary data center load balancing represents a complex engineering challenge encompassing multiple levels, perspectives, and scenarios. In addition to the aforementioned approaches, this section examines load balancing services offered by major cloud service providers, including AWS, Azure, AlibabaCloud, and Google.

The evolution of cloud computing load balancing technology mirrors the progress of the entire industry. Early load balancing primarily focused on basic traffic distribution, but has now evolved into an intelligent scheduling hub for data centers. During this evolution, major

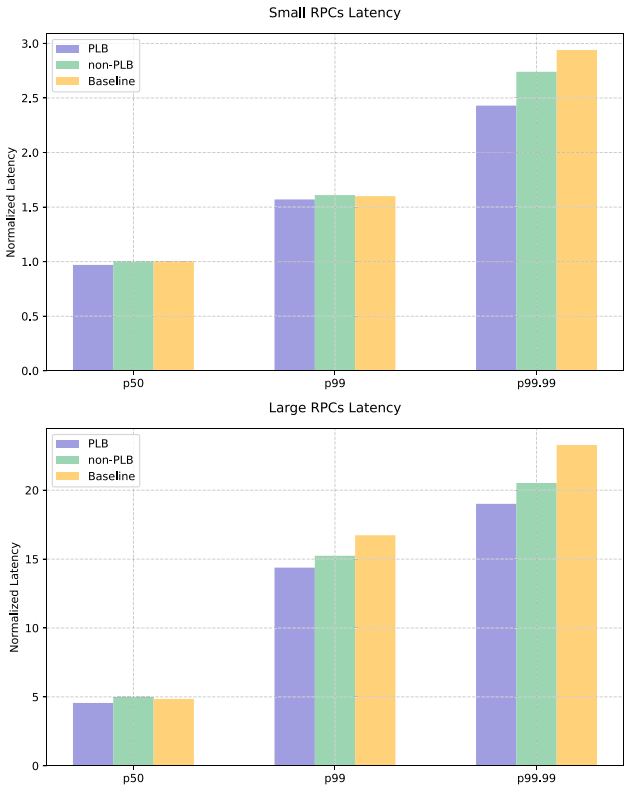


Fig. 9. Normalized transfer latencies with 20% PLB and 80% non-PLB traffic.

cloud service providers have developed unique technical schemes based on their technical expertise and market positioning.

As a pioneer in cloud computing, AWS's load balancing services demonstrate this development journey. Starting with the Classic Load Balancer, AWS has focused on providing reliable traffic scheduling schemes. With the rise of container technology and microservice architecture, AWS introduced the Application Load Balancer specifically for HTTP/HTTPS traffic, resolving traffic scheduling challenges in microservice architectures through deep integration with container services. The launch of Network Load Balancer addressed the demands of latency-sensitive scenarios such as financial trading and gaming, with its millisecond-level response capability providing crucial support for such applications.

Microsoft Azure chose a different technical scheme. Building on its strong foundation in the enterprise market, Azure's load balancing schemes emphasize integration with existing enterprise IT architectures. Its Layer 4 load balancer, with basic and standard tiers, caters to both simple requirements of small businesses and high availability needs of large enterprises. Notably, Azure Application Gateway integrates Web Application Firewall functionality, serving both Layer 7 load balancing and application security protection duties.

Alibaba Cloud's development strategy reflects the characteristics of the local market. Its Server Load Balancer (SLB) product line adopts a progressive evolution scheme, serving existing customers with traditional load balancers while launching next-generation balancers for containerized applications. Particularly in handling high-concurrency scenarios, its Network Load Balancer (NLB) demonstrates excellent performance, providing reliable support for extreme scenarios like e-commerce peak sales events.

Google Cloud's scheme is entirely different, building a unique distributed load balancing system based on its global data center infrastructure. Through Software-Defined Networking (SDN) technology, requests are first directed to the nearest point of presence, then intelligently routed through the global network to the optimal data

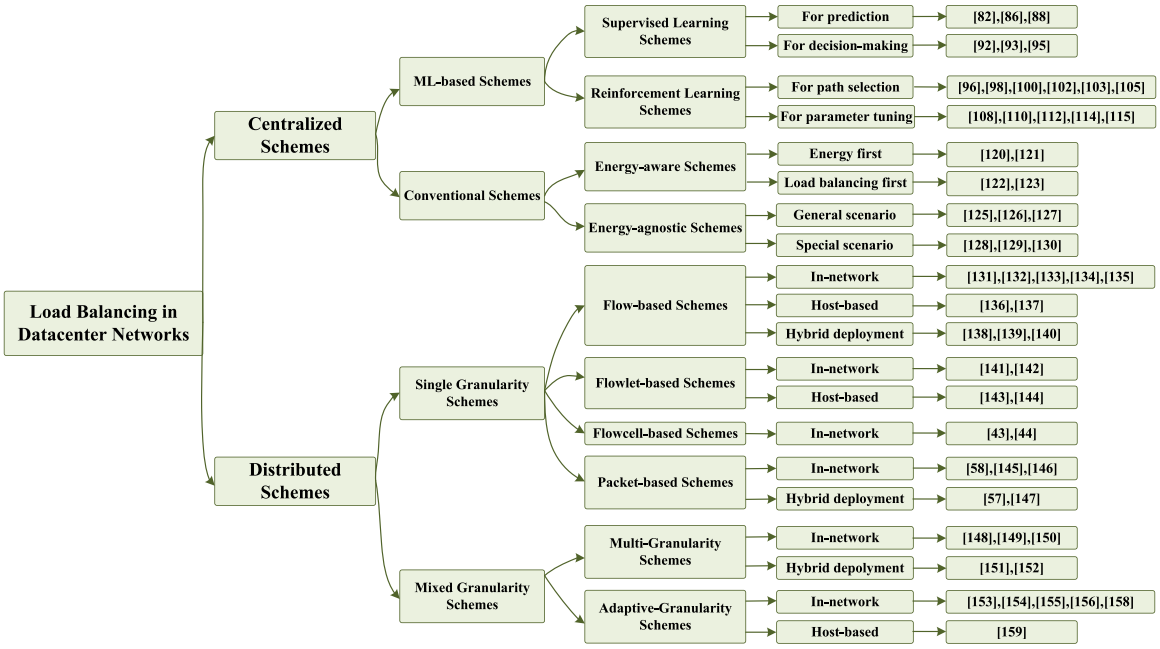


Fig. 10. Categorization of the reviewed literature.

center. This proxy-free distributed architecture not only provides exceptional performance but also offers flexible support for cross-region deployment.

These different technical schemes reflect each cloud provider's strategic priorities: AWS emphasizes service completeness and advancement, Azure focuses on deep enterprise market integration, Alibaba Cloud prioritizes performance optimization and scenario adaptation, while Google Cloud pursues global reach and architectural innovation. Moving forward, as new technologies like edge computing and 5G develop, load balancing services will continue to evolve, providing more intelligent scheduling capabilities while ensuring performance.

2.5.3. Session-aware load balancing

When discussing data center load balancing technologies, session-aware load balancing deserves significant attention. Although it falls under the category of server load balancing, it has become increasingly crucial in contemporary data center load balancing practices.

Session-aware load balancing primarily operates at the application layer (Layer 7) and originated in the field of web server load balancing. Its core idea is to fully consider the session to which a request belongs when distributing requests, directing requests from the same session to the same server or service instance. This capability is vital for guaranteeing user experience and service continuity, especially in stateful application scenarios such as social media and online conferences.

As technology evolves, session-aware load balancing has also been introduced into the management and orchestration of virtual network functions (VNFs). Deploying it as a key VNF component can significantly improve the performance and scalability of stateful network functions in VNF environments. This is particularly important in today's era of increasingly prevalent cloud services.

In terms of implementation, session-aware load balancing generally follows two approaches: hardware load balancers and software load balancers. The former utilizes built-in dedicated hardware to achieve session-aware scheduling based on the transport layer (L4) and load distribution at the application layer (L7) simultaneously. However, they are usually expensive and have limited scalability. The latter exists in the form of software, with open-source schemes such as Nginx [76], LVS [77], and HA-Proxy [78] serving as key examples. They can be flexibly deployed in service chains as VNFs. Although their performance may not match that of hardware load balancers, they can meet load balancing demands with lower costs and higher elasticity.

In recent years, with the rise of P4 programmable data planes, cutting-edge research such as Yoda [79], Prism [80], and Yuz [81] has begun exploring the offloading of session state and scheduling policies to programmable switches and smart NICs. The goal is to achieve higher-performance and finer-grained session-aware traffic scheduling. This indicates the direction of further development in session-aware load balancing technology.

Understanding and addressing these challenges is essential for designing effective load balancing strategies and optimizing overall performance in modern data center networks. The analysis is based on a comprehensive review of approximately sixty papers published in major journals and conferences within the past five years. These papers were systematically categorized according to the classification presented in Fig. 10, with detailed analyses and comparisons provided in Sections 3 and 4.

3. Centralized schemes of load balancing in data center networks

Centralized approaches typically rely on a central controller to collect and process global network state information. All decisions are made from a centralized, global perspective, endowing these approaches with an inherent global view that enables globally optimal scheduling decisions. As network scale and complexity continue to increase, the application of intelligent technologies for load balancing decisions has garnered increasing attention. In data center networks, controllers possess substantial processing capabilities and aggregate vast amounts of network state data, including traffic matrices and link utilization metrics. Artificial intelligence techniques, such as machine learning and reinforcement learning paradigms, leverage data-driven methods to adaptively learn and predict network states from this wealth of information, thereby guiding load balancing decisions. This approach aligns well with the architecture of centralized schemes, capitalizing on both high processing power and abundant data availability. Consequently, machine learning technologies have begun to empower load balancing strategies. The subsequent sections examine centralized approaches from two perspectives: machine learning-based and conventional methods, followed by a comprehensive discussion of centralized schemes. A summary of centralized approaches is presented in Table 3.

Table 3
Comparison of centralized load balancing schemes.

Scheme	Category	Performance	Limitations	Suggested improvements
LLBR [82]	Supervised learning	50% less transmission time, 30% better bandwidth use	Suffers from high computation overhead and relies on historical data	Should develop lighter model and add real-time updates
HMMLB [83]	Supervised learning	50% faster path selection, comparable throughput to centralized	Shows poor inference accuracy and depends on historical data	Needs to improve inference and add adaptive mechanisms
LLB [84]	Supervised learning	Outperforms ECMP, 120 Gbps FPGA inference	Requires complex model and large training dataset	Should simplify model and optimize hardware implementation
ANN-KDN [85]	Supervised learning	Higher throughput vs. ECMP and random	Places heavy load on controller and has prediction issues	Needs to reduce controller load and enhance prediction
BULB [86]	Supervised learning	175x faster decisions, 2% performance loss	Demands high training cost and complex update process	Should optimize training and simplify update procedure
Babayigit [87]	Supervised learning	2 ms response (vs. 18 ms SVM), 92% accuracy	Lacks sufficient testing and scalability validation	Needs more testing and scalability proof
DeepRLB [88]	Reinforcement learning	45.1%/35.3% lower average/tail FCT vs. ECMP	Suffers from long training time and unstable convergence	Should improve training stability and reduce overhead
Tosounidis [89]	Reinforcement learning	Outperforms Round Robin and handles 40% more requests	Shows high computational cost and complex system integration	Needs to simplify architecture and reduce complexity
GDLB [90]	Reinforcement learning	7%/11.2% lower max link use vs. ECMP	Requires high computational resources and limited topology support	Should enhance scalability and reduce resource usage
DRLB [91]	Reinforcement learning	13.74% higher throughput, 3.18% lower delay vs. ECMP	Faces adaptation issues in dynamic environments and high overhead	Needs to improve adaptability and reduce overhead
DRL-TAL [92]	Reinforcement learning	No packet loss, 45% lower FCT vs. CONGA	Demands complex parameter tuning and accurate network state info	Should simplify parameter tuning and enhance robustness
SPRLN [93]	Reinforcement learning	30% higher throughput, 80% less data loss	Shows high control overhead and complex implementation	Needs to reduce control overhead and simplify design
EDDQN [94]	Reinforcement learning	8.3% higher average link utilization vs. ECMP	Requires extensive monitoring and complex elephant flow detection	Should optimize detection and reduce monitoring cost
RILNET [95]	Reinforcement learning	30.9% lower MLU, 44.4% less packet loss vs. DRILL	Demands high training overhead and strict real-time requirements	Needs to improve training efficiency and response time
RWCMP [96]	Reinforcement learning	24%–36% lower network use vs. LetFlow	Shows convergence issues and potential routing instability	Should enhance stability and improve convergence rate
DRL-Plink [97]	Reinforcement learning	Outperforms ECMP/pFabric/Karuna in FCT and deadline rates	Faces complex implementation and high system requirements	Needs to simplify implementation and reduce complexity
AggreFlow [98]	Energy-aware	18% power saving, 98% less control messages	Shows coarse-grained balancing and high delay sensitivity	Should improve granularity and reduce delay impact
BiTE [99]	Energy-aware	Outperforms TPM/NSGA-II/ElasticTree in link utilization	Suffers from high algorithm complexity and slow convergence	Needs to simplify optimization and speed up convergence
ESLB [100]	Energy-aware	Better throughput and link utilization vs. DLB/NLB	Requires complex ILP solving and shows poor scalability	Should improve solving efficiency and enhance scalability
Pathan [101]	Energy-aware	Superior energy efficiency and flow success rates	Faces high computation complexity in MILP formulation	Needs to reduce complexity and optimize runtime
SAPS [102]	Energy-agnostic	Improved FCT under data mining and web search workloads	Shows limited support for asymmetric topologies	Should enhance topology support and improve adaptability
PSO [103]	Energy-agnostic	Better bandwidth use and throughput vs. ECMP/SA	Demands high control overhead and complex parameter tuning	Needs to reduce overhead and simplify parameter settings
IALB [104]	Energy-agnostic	Higher link utilization and throughput vs. ECMP/Hedera	Requires frequent network state collection and monitoring	Should optimize monitoring and reduce state collection
Zheng [105]	Energy-agnostic	12% reduction in traffic congestion	Shows limited adaptation to dynamic traffic patterns	Needs to improve dynamic adaptation and flexibility
Chung [106]	Energy-agnostic	44.1% throughput improvement in random traffic	Faces scalability issues in large-scale deployments	Should enhance scalability and reduce complexity
Oddlab [107]	Energy-agnostic	30%–62% FCT reduction vs. ECMP/Hedera/PureSDN	Shows detection accuracy issues and high response delay	Needs to improve detection and reduce response time

3.1. ML-based schemes

The application of machine learning in data center network load balancing has grown with the advantages of machine learning algorithms in handling complex, dynamic systems. This section will explore two main machine learning approaches: supervised learning and reinforcement learning.

Section 3.1.1 will introduce schemes based on supervised learning, which are suitable for network environments with relatively stable traffic patterns. Supervised learning schemes fall into two categories: those focused on network traffic prediction and those dedicated to decision-making in load balancing strategies. Prediction-based algorithms forecast traffic based on historical data, while decision-making algorithms make load balancing decisions based on known network states.

Section 3.1.2 will discuss reinforcement learning schemes. Since reinforcement learning involves decision-making through interaction with the environment, reinforcement learning approaches comprise two

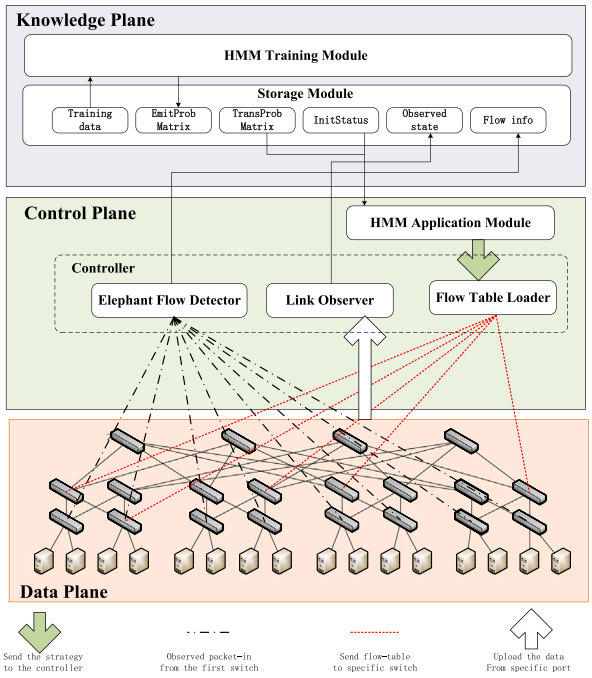


Fig. 11. The architecture of HMMLB.

categories: those that directly participate in path selection decisions, and those that optimize parameter tuning. The first category involves algorithms that select the optimal path in real-time, while the second optimizes other network parameters, indirectly influencing load balancing performance.

Each approach has its own characteristics and is suited for different scenarios. This analysis provides better understanding of their applications in load balancing and their potential for optimizing decision-making.

3.1.1. Supervised learning schemes

3.1.1.1. Supervised learning for prediction. LLBR [82] uses deep learning to predict link utilization distribution for the next cycle, guiding current load balancing routing decisions. It employs a residual network-based spatio-temporal data-aware link utilization prediction model ST-ResNet [108] and combines prediction results with Dijkstra's algorithm to optimize path selection based on estimated costs in each scheduling cycle. Experiments conducted on a K = 8 Fat-Tree topology with 128 hosts and 80 switches demonstrate that LLBR achieves significant performance improvements in transmission time, bandwidth utilization, delay jitter, and packet loss rate compared to RRS [109] and WRRS [110] algorithms.

HMMLB [83] transforms the path selection problem into a Hidden Markov Model problem [111]. It selects only a subset of parallel links for monitoring and infers the state of the entire path from these monitored links. The algorithm is trained using supervised learning on data collected from traditional centralized algorithms like Hedera's First Fit and Greedy strategies. HMMLB achieves comparable network throughput to traditional centralized algorithms while significantly reducing the number of monitored links and path selection time. Simulation experiments on fat-tree topologies demonstrated that HMMLB could reduce the average time allocated to each flow by nearly half compared to Hedera, validating its effectiveness (see Fig. 11).

ANN-KDN [85] leverages ANN within a Knowledge-Defined Networking [112] framework for data center load balancing. It addresses the limitations of traditional methods that use specialized hardware, which are costly and inflexible. The ANN-KDN framework utilizes SDN's centralized programmable control capabilities and ANN's modeling abilities to predict network performance based on traffic parameters such as bandwidth and latency, learning to select paths with minimal load. The method is implemented using Mininet [113] as the network emulator and OpenDaylight [114] as the SDN controller, simulating a k-ary Fat-Tree topology. Performance evaluation demonstrates significant improvements in throughput compared to ECMP and Random load balancing methods, suggesting the potential of integrating machine learning techniques with SDN for more efficient network management in data centers.

3.1.1.2. Supervised learning for decision making. LLB [84] is a universal learning-based load balancing framework designed to adaptively learn optimal strategies for various topologies, traffic patterns, and failure scenarios. It uses deep supervised learning on synthetic data generated from distributions modeled after real data center traffic statistics to train a load balancing model capable of handling topology changes and link failures. The core approach involves solving a multi-commodity flow engineering optimization problem using global telemetry information to obtain optimal path weights as training labels. A three-layer feedforward neural network with 128 hidden units per layer is then designed to learn the mapping from traffic matrices to path weights. The deployment utilizes smart NICs for real-time traffic matrix measurement and programmable switches for packet forwarding, achieving significant performance advantages over heuristic and adaptive algorithms like ECMP.

Babayigit et al. [87] propose a deep learning based method for addressing load balancing issues in SDN data center networks. The research uses a k-ary fat-tree topology to construct the network, with k set to 4, and employs Floodlight [115] as the SDN controller and Mininet as the simulation platform. The system obtains path cost information through Rest-API, calculating link load based on transmitted and received data volumes. The researchers designed a four-layer deep learning model, trained using dynamic link load data. This model can determine optimal load balancing paths in real-time and was tested in two different load balancing scenarios. Experimental results show that this method outperforms the SVM algorithm in response speed, with an average response time of about 2 ms compared to SVM's 18 ms. In terms of accuracy, the method surpasses the ANN approach, achieving an accuracy of approximately 92% compared to ANN's 67%, demonstrating balanced overall performance.

BULB [86] addresses datacenter load balancing challenges by innovatively separating complex learning from quick online decisions. It employs a DDPG algorithm offline to train a DRL agent that optimizes link weights based on network state information. To bridge the gap between sophisticated DRL models and real-time requirements, BULB converts the DRL agent's neural network into a lightweight decision tree using imitation learning with data resampling. This decision tree is deployed on a central controller for periodic link weight updates, enabling quick, localized routing decisions at end-hosts. Simulations with realistic workloads demonstrate BULB's effectiveness: in asymmetric topologies with web search traffic, it reduced average and tail flow completion times for all flows by 45.1% and 35.3% compared to ECMP and Hermes, and outperformed existing schemes across all flow sizes for memcached workloads. The decision tree conversion achieved a 175x reduction in decision latency with only a 2% performance loss, highlighting BULB's ability to balance efficiency and effectiveness in datacenter load balancing (see Fig. 12).

3.1.2. Reinforcement learning

3.1.2.1. Reinforcement learning for path selection. SPRLN [93] proposes an intelligent load balancing scheme based on reinforcement learning, specifically designed for software-defined data center networks. The core of this scheme is the adoption of the Q-learning [116] algorithm to continuously optimize path selection strategies, adapting to complex and dynamic traffic patterns. The SPRLN architecture comprises four

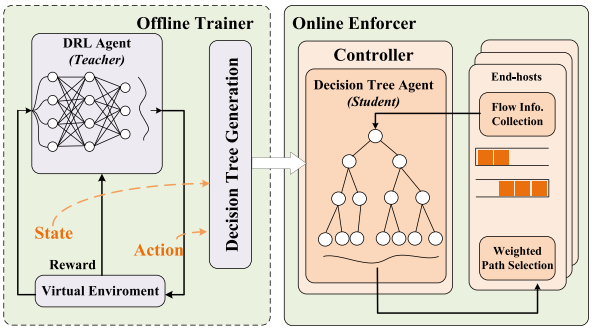


Fig. 12. BULB structure.

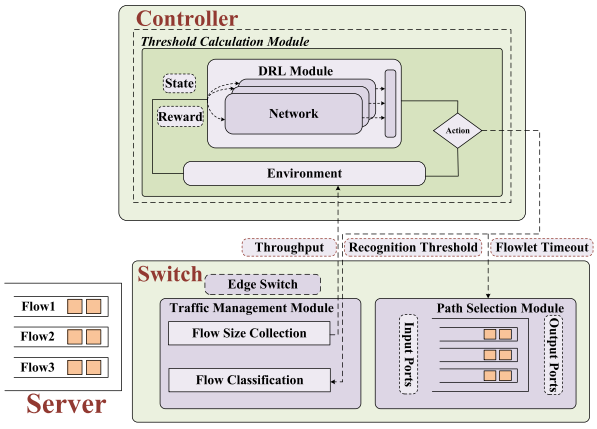


Fig. 13. DRL-TAL structure.

key components: a learning module, prediction data storage, prediction algorithm, and network traffic discovery, forming a self-optimizing closed-loop system. In the specific application of Q-learning, SPRLN defines switch utilization and throughput as states, path selection between nodes as actions, and calculates rewards based on link costs. The scheme employs different strategies for elephant and mice flows: implementing flow slicing and multi-path allocation for elephant flows, while using simpler routing for mice flows. By introducing a splitting policy, SPRLN effectively reduces the need for polling the SDN controller, thereby lowering control plane overhead. Experimental results demonstrate that SPRLN achieves significant improvements in multiple network performance metrics compared to existing algorithms, including reduced RTT, 30% increase in throughput, and 80% reduction in data loss rate.

Tosounidis et al. [89] combines Convolutional Neural Networks and Q-learning for load balancing in SDN environments. DQN [117] uses CNN to automatically extract key features from raw network monitoring data, overcoming limitations of traditional tabular reinforcement learning. It controls request scheduling at the (server, duration) granularity and optimizes routing strategies through a cost function that combines immediate and long-term rewards. DQN operates as an independent agent interfacing with the SDN controller via northbound APIs, reducing the controller's computational burden while maximizing machine learning flexibility. The model's architecture includes a CNN for processing network state data, a separate NN for server load information, and a final NN for decision-making. Experiments on a fat-tree topology simulated in Mininet demonstrate DQN's superior performance over traditional Round Robin and Weighted Round Robin algorithms, handling 40% more requests and reducing response times by over 50%. This approach showcases adaptability to dynamic network conditions and the ability to detect hidden patterns in complex SDN environments.

RILNET [95] uses the DDPG algorithm to learn network states and make globally optimal routing decisions. It employs aggregated traffic throughput time series as input states, processed through a convolutional neural network to extract spatiotemporal features of traffic changes. This method makes routing decisions at the aggregated flow level and uses flowlets as internal forwarding units. RILNET's reward function aims to minimize the variance of path utilization, explicitly modeling network congestion. In terms of architectural design, RILNET consists of three main components: edge switches, data collectors, and AI controllers. The neural network structure includes a CNN layer, a pooling layer, and three fully connected layers, trained using the ADAM optimizer [118]. In experimental evaluations, RILNET demonstrates superior load balancing performance compared to ECMP and DRILL, particularly excelling in asymmetric topologies.

EDDQN [94] focuses on detecting and scheduling elephant flows, which significantly impact network performance. It introduces a two-stage detection method for elephant flows, operating at switch and controller levels. The first stage identifies suspicious flows at switches using duration-based thresholds, while the second stage at the controller applies rate-based criteria. This approach filters most mice flows at the switch, reducing controller-switch communication. For rerouting, EDDQN employs a customized DQN model considering network topology, traffic patterns, and bandwidth fragmentation. The DQN's reward function optimizes packet loss, throughput, and resource utilization. Dynamic periodic sampling adjusts intervals based on network congestion, enhancing efficiency. The model incorporates a bandwidth fragmentation handling module to improve link resource usage. Experiments show EDDQN outperforms ECMP and Hedera in throughput, link utilization, and delay reduction across various traffic loads, demonstrating its adaptability in complex data center networks.

GDLB [90] innovatively combines Graph Convolutional Networks [119] with DRL for adaptive traffic scheduling. GCN efficiently extracts key features from network topology and state information, while DRL optimizes routing strategies through sequential decision-making. The model's architecture consists of three layers of GCN and two layers of CNN, allowing for comprehensive feature extraction and processing. GDLB utilizes historical network states to capture temporal correlations in traffic patterns. GDLB's reward function considers multiple performance indicators, including maximum link utilization, delay, and packet loss rate, aiming for load balancing while ensuring QoS. Simulation experiments demonstrate GDLB's superior performance compared to existing AI methods like Deep Q-Learning and SRL. The algorithm's performance advantage becomes more pronounced in larger traffic scenarios, making it particularly suitable for managing complex data center networks with dynamic and uncertain traffic conditions.

DRLB [91] addresses the limitations of traditional load balancing strategies in heterogeneous traffic environments. It employs different strategies for long and short flows: using D4PG [120] for optimal path decision-making for long flows, and WCMP routing for short flows. This approach reduces short flow delay and increases long flow throughput. DRLB uses queue length and throughput as state representations, RTT of all links for reward calculation, and forwarding decisions as actions. The mechanism dynamically adjusts the Link State Metric to guide the forwarding of long flows, ensuring adaptive load balancing in rapidly changing network conditions. DRLB's performance was evaluated using NS-3 simulations under various traffic workloads and topology scenarios, demonstrating significant improvements over existing mechanisms like Freeway [121], Hermes, and TLB. Notably, DRLB reduced the 99th percentile FCT of short flows by up to 58% and increased the throughput of long flows by up to 47% compared to these existing mechanisms.

3.1.2.2. Reinforcement learning for parameter tuning. RWCMP [96] treating traffic splitting ratios as stochastic policies and learning optimal load balancing strategies by minimizing network resource utilization.

The method employs a centralized control architecture, using the PPO [122] algorithm for policy optimization and combining it with an LSTM network model to capture long-term sequence information. RWCMP's state space includes demand sequences and link utilization sets, while the reward function is based on overall network utilization and maximum link utilization. To avoid packet reordering, the algorithm uses flowlet-level scheduling granularity and prevents routing loops by truncating links that could lead to routing cycles. Compared to traditional load balancing algorithms like ECMP and LetFlow, RWCMP can adaptively learn traffic splitting ratios without manual parameter tuning, performing well under various traffic patterns and network topologies, including scenarios with link failures.

Designed for mobile edge computing scenarios, TRO [123] addresses low bandwidth utilization and high latency issues in edge data centers. It combines SDN's global control characteristics with deep reinforcement learning's perception and decision-making capabilities. TRO employs the TD3 (Twin Delayed Deep Deterministic Policy Gradient) algorithm, an improvement over DDPG, using network state information as input and link weights as actions. The reward function is a composite of two components: a positive reward based on average occupied bandwidth, encouraging higher throughput, and a negative reward derived from the difference between maximum and minimum traffic rates, promoting load balancing across the network. This dual-aspect reward mechanism enables TRO to optimize both overall network utilization and traffic distribution simultaneously. The approach demonstrates excellent performance in Mobile Edge Computing (MEC) [124] environments, effectively enhancing network performance in terms of throughput, latency, and load balancing.

DeepRLB [88] deploys a DDPG [125] agent in the SDN controller to adaptively learn link weight values based on traffic characteristics. It consists of a forwarding management module and a statistics management module. The state space includes traffic demand matrices at each time step, actions are link weights, and the reward function is the negative variance of link utilization to achieve load balancing across paths. Unlike traditional approaches that use the maximum link utilization metric, DeepRLB adopts variance as the optimization objective to more comprehensively reflect network load distribution. The method leverages the advantages of deep reinforcement learning to adapt to dynamic data center traffic patterns and performs well in various network topologies such as Fat-tree and VL2. By optimizing link weights DeepRLB can effectively allocate traffic to optimal paths, thereby improving network performance and reducing congestion.

DRL-Plink [97] is a load balancing framework designed for softwar e-defined data center networks. It deploys a DDPG algorithm on the SDN controller, establishing private links for different traffic types including mice flows MF, elephant flows EF1 and EF2, and dynamically adjusting bandwidth allocation. DRL-PLink introduces techniques such as clipped double Q-learning, exploration with noise, and prioritized experience replay to enhance DDPG performance. Simulation experiments conducted on a 4-pod fat-tree topology using real data center workloads of Web Search and Data Mining demonstrate that DRL-PLink significantly reduces average and tail flow completion times FCT compared to existing schemes like ECMP, pFabric, and Karuna, while also improving deadline meet rates DMR. Additionally, DRL-PLink introduces the Gini coefficient to evaluate load balancing performance, proving its effectiveness in achieving load balance between paths.

DRL-TAL [92] dynamically adjusts flowlet timeout thresholds using deep reinforcement learning. This innovative load balancing scheme for data center networks comprises three key modules: traffic management, path selection, and DRL-based threshold calculation. The traffic management module continuously monitors edge switch throughput, collecting real-time network state information. For path selection, DRL-TAL employs different strategies based on flow size: packet-level granularity for mice flows and flowlet-level granularity for elephant flows. At its core, the threshold calculation module utilizes the DDPG algorithm to adaptively adjust flowlet timeout thresholds. By constantly sensing network throughput and generating optimized thresholds for each subsequent time period, DRL-TAL can dynamically adjust its behavior to match changing network conditions, resulting in more efficient load balancing. Experiments conducted on a Fat-tree topology demonstrate DRL-TAL's superior performance across various load scenarios, particularly in high-load and asymmetric network configurations (see Fig. 13).

3.2. Conventional schemes

Despite the growing popularity of machine learning methods, conventional load balancing schemes remain dominant in production environments and continue to evolve. These traditional approaches, based on classical algorithms and heuristic methods, offer key advantages in interpretability and computational efficiency.

Building upon Zhang et al.'s work [10], we adopt and refine their energy-based classification for traditional load balancing schemes. This dimension remains significant, as data centers currently consume 1.5% of global electricity [126], with projections indicating an increase to 4%–8% by 2030 [127]. Major tech companies' commitments further underscore this importance, with Google [128] targeting net-zero carbon emissions in global operations by 2030 and Amazon [129] by 2040.

Specifically, we refine the taxonomy of traditional schemes by further categorizing both energy-aware and energy-agnostic approaches based on their design objectives:

Energy-aware schemes comprise two subcategories based on their optimization priorities. Energy-first schemes focus primarily on reducing energy consumption while meeting basic performance requirements. In contrast, load-balancing-first schemes prioritize load balancing objectives while achieving energy optimization through balanced resource allocation.

Energy-agnostic schemes divide into general-scenarios and special-scenarios approaches. General-scenarios schemes address common load balancing requirements, emphasizing system performance and resource utilization. Special-scenarios schemes target specific application contexts or requirements with specialized optimization objectives.

This classification framework enables a comprehensive analysis of traditional load balancing techniques, illuminating current trends and application-specific considerations.

3.2.1. Energy-aware schemes

3.2.1.1. Energy-first schemes. ESLB [100] introduces an innovative traffic scheduling strategy for data center networks, simultaneously addressing load balancing and energy conservation objectives. The strategy formulates the problem as an Integer Linear Programming (ILP) issue, fully utilizing the multi-path characteristics of Fat-tree topologies. ESLB employs a two-step path selection process: first identifying the most energy-efficient path set, then choosing the path with the lowest average link utilization. To tackle the high computational complexity of the ILP problem, the authors propose a heuristic algorithm. In handling traffic, ESLB adopts a multi-path routing approach, allocating excess traffic to alternative paths when demand exceeds preset link capacity thresholds. Performance evaluations demonstrate that ESLB maintains high network throughput across various traffic loads, slightly outperforming the PPBG algorithm in energy optimization and significantly surpassing the shortest path algorithm. In terms of load balancing, ESLB outperforms DLB and NLB algorithms in most scenarios, exhibiting superior network resource utilization and higher throughput. Notably, when traffic volume is lower, ESLB shows lower link utilization and higher throughput rates, proving its effectiveness in achieving both energy efficiency and performance optimization in data center networks.

AggreFlow [98] addresses two critical issues in data center networks: unbalanced traffic distribution in energy-efficient networks and the burden of frequent control messages on OpenFlow switches during

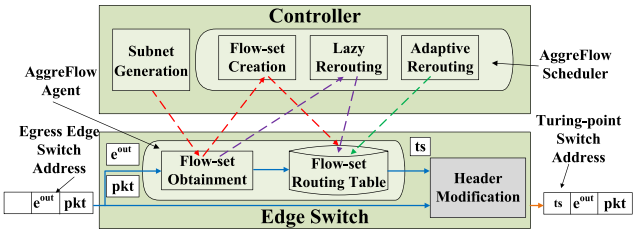


Fig. 14. AggreFlow structure.

subnet changes. It introduces three core technologies: Flow-set Routing, which aggregates flows with identical hash values for coarse-grained load balancing; Lazy Rerouting, which delays rerouting operations to spread the control message load over time; and Adaptive Rerouting, which selectively adjusts flow paths to maintain load balance. Through these innovations, AggreFlow achieves significant improvements in energy efficiency, load balancing, and network performance. Simulation results demonstrate its ability to save 18% power while maintaining good load distribution, drastically reducing control message overhead by 98%, and ensuring low packet loss rates, making it more comprehensive for modern data center network management challenges (see Fig. 14).

3.2.1.2. Load-balancing-first schemes. BiTE [99] presents an innovative bi-level traffic engineering mechanism for data center networks, aiming to optimize traffic allocation while balancing network performance, load distribution, and energy efficiency. It models the traffic allocation as a multi-cycle, bi-level optimization problem, where the upper level focuses on load balancing using an ant colony algorithm, and the lower level targets energy efficiency through a simulated annealing algorithm. This approach allows for dynamic adaptation to traffic changes, with the SDN controller periodically triggering optimization. Simulation results demonstrate BiTE's effectiveness in achieving a balance between load distribution and energy efficiency, particularly in large-scale data center networks, offering a comprehensive approach to the complex challenges of modern network management.

Pathan et al. [101] presents novel routing schemes for SDN-enabled data center networks that jointly optimize flow priority, energy consumption, and load balancing. The authors first develop a Mixed Integer Linear Programming [130] formulation for optimal routing, then propose two greedy algorithms to address MILP's potential runtime issues: Priority Based Energy Minimization Algorithm (PEMA) and Priority Based Evenly Load Distribution Algorithm (PEDL). These schemes employ an Energy Aware Path Finding method to identify efficient paths and a Load Capacity Calculation method to ensure link capacity constraints are met. PEMA focuses on energy-efficient paths, while PEDL aims for even load distribution. Simulations on a fat-tree topology demonstrated that PEMA achieved superior energy savings, while PEDL excelled in load balancing. Both outperformed existing schemes in flow success rates and combined optimization, highlighting the effectiveness of these comprehensive schemes for DCN routing.

3.2.2. Energy-agnostic schemes

3.2.2.1. General-scenarios schemes. To address elephant flow conflicts, DMLBP [103] proposes a dynamic multi-path load balancing algorithm based on particle swarm optimization. The algorithm periodically acquires global network information. The controller calculates the average and standard deviation of flow byte counts within time T to determine the threshold. Topology information is collected via LLDP packets sent through *Packet_out* messages. Switches forward these packets, and neighboring switches send *Packet_in* messages for unmatched packets. The controller then parses and records topology information for a global view. The particle swarm optimization algorithm represents path scheduling schemes for large flows as particle positions, with each

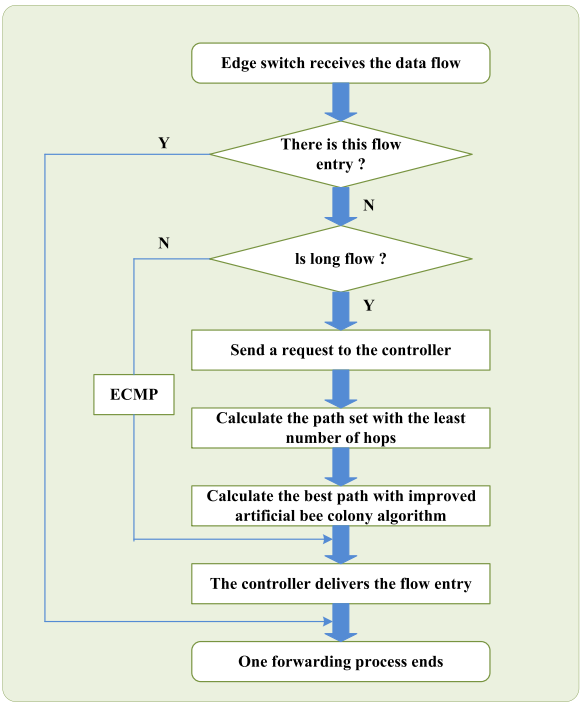


Fig. 15. The flow diagram of IALB.

dimension corresponding to path numbers for large flows. The fitness function uses the load balancing factor, defined as the variance of network bandwidth utilization. Particles update their positions based on individual and global best fitness values, with the global optimum representing the optimal path for large flows.

To address link congestion and load imbalance issues caused by traditional ECMP routing in data center networks, as well as the additional overhead of existing improvements like Hedera for monitoring large flows at the forwarding layer, the IALB [104] is proposed. Operating within an SDN architecture, IALB utilizes the global network view maintained by the centralized controller to dynamically schedule large flows. It uses ECMP to handle small flows, while triggering the IALB algorithm for detected large flows. IALB is based on an improved artificial bee colony algorithm, overcoming the tendency of traditional bee colony algorithms to fall into local optimal. Compared to existing methods, IALB achieves more efficient and intelligent traffic scheduling optimization without increasing network overhead. It reduces maximum link utilization, improves network throughput, and enhances load balancing (see Fig. 15).

Asymmetric topologies in large-scale data center networks pose significant challenges to effective load balancing. These asymmetries, commonly arising from link performance degradation or switch failures, can lead to inefficient resource utilization and compromised network performance. Traditional approaches such as weighted packet spraying often underperform in heterogeneous scenarios due to their inability to effectively cooperate with TCP congestion control mechanisms. To address these issues, this study introduces SAPS [102], a novel load balancing scheme specifically designed for asymmetric topologies in DCNs. SAPS employs a innovative approach that performs packet spraying under conditions of symmetric bandwidth and latency. The algorithm utilizes a centralized controller for fault detection and constructs virtual topologies to manage traffic distribution. Large flows are strategically mapped to high-bandwidth virtual topologies, while small flows are randomly mapped based on probabilistic mechanisms. This method enables SAPS to effectively utilize links with reduced bandwidth by constructing low-bandwidth symmetric topologies, thereby enhancing overall network resource utilization. Unlike

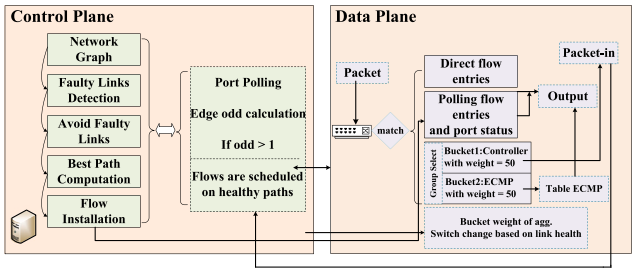


Fig. 16. Oddlab structure.

approaches that simply avoid using faulty links, SAPS leverages partially degraded links to maintain network symmetry and improve performance. This adaptive strategy not only enhances load balancing in asymmetric scenarios but also demonstrates superior compatibility with TCP congestion control mechanisms, addressing a critical limitation of existing approaches in heterogeneous network environments.

3.2.2.2. Special-scenarios schemes. Zheng et al. [105] delves into the dynamic load balancing problem in hybrid switching data center networks equipped with converters. The researchers developed an innovative approach by designing topology-specific converters and proposing an optimization framework for the dynamic load balancing problem. Through rigorous theoretical analysis, they established the problem's NP-hardness and ρ-inapproximation properties. To tackle this challenge, a two-stage greedy algorithm was devised, incorporating flow re-routing and converter reconfiguration to minimize maximum link utilization. Extensive simulations across various network topologies demonstrated significant performance improvements, including a 12% average reduction in traffic congestion and rapid convergence within three iterations. The algorithm's effectiveness was validated under both uniform and heavy-tailed traffic distributions, highlighting its versatility. This research not only advances the theoretical understanding of load balancing in complex network environments but also provides a practical strategy with broad applicability, potentially revolutionizing data center network management in the face of diverse and dynamic traffic patterns.

Oddlab [107] proposes a fault-tolerant load balancing framework for data center networks, focusing on addressing link failure issues through an innovative detection mechanism that analyzes switch load ratios and link throughput correlations. The system collects comprehensive network information from the data plane to construct a heuristic algorithm model for optimizing traffic distribution across multiple paths. Fault detection employs a periodic polling mechanism, monitoring all switch port statistics and marking links as faulty after three consecutive polling cycles below 1% utilization threshold, and as long-term failures after five cycles. The path selection algorithm prioritizes links with the lowest elephant flow count and highest available bandwidth under normal conditions (odd ≤ 1), while in heavy load scenarios (odd > 1), it first excludes faulty links before selecting the optimal path, ensuring efficient traffic distribution even in the presence of network failures (see Fig. 16).

In BCube data center networks, traditional distributed routing algorithms like BSR often lead to inefficient resource utilization due to lack of coordination. To address these limitations and enhance load balancing efficiency, Chung et al. [106] proposes two innovative centralized scheduling algorithms: Centralized Dynamic Parallel Flow Scheduling (CDPFS) and its multi-path variant CDPFSMP. These algorithms leverage a centralized control paradigm, incorporating a global network view to make informed routing decisions. CDPFS employs a strategic partitioning of the BCube topology into disjoint subgraphs, facilitating parallel path computation for efficient flow distribution. CDPFSMP extends this concept by exploiting the multi-interface architecture of BCube servers, enabling the splitting of large flows across multiple disjoint paths. This approach not only maintains the bandwidth efficiency of single-path transmission but also significantly mitigates link overload, thereby enhancing overall network throughput. By dynamically allocating paths based on real-time flow sizes and network conditions, these algorithms demonstrate a marked improvement in load balancing and network performance. The study's findings underscore the potential of centralized, adaptive routing strategies in addressing the complex traffic management challenges inherent in modern data center networks.

3.3. Discussion of centralized schemes

This section examines centralized load balancing schemes, categorized into machine learning-based approaches and traditional methods. This classification demonstrates both the evolution of data center network load balancing technology and the distinct advantages and challenges each technological approach presents in addressing complex network environments.

Machine learning-based schemes mainly include supervised learning and reinforcement learning based approaches. Supervised learning methods, such as ANN-KDN and LLBR, guide load balancing decisions by predicting future traffic trends, showing strong predictive capabilities and the ability to capture complex patterns. Since the behavior of the model after training is relatively predictable, supervised learning usually has good stability, but the problem is that supervised learning requires a large number of high-quality labeled data to learn, but the acquisition of high-quality labeled data itself is a relatively difficult thing, when the network environment changes dramatically, the performance of the model may rapidly decline. Reinforcement learning methods, such as SPRLN and GDLB, focus on making optimal decisions based on the current state of the network, do not need to be trained on pre-labeled datasets, and show strong adaptability, but the problem is that their stability and interpretability are poor, and it takes a long time to converge to a better strategy.

Traditional approaches can be divided into energy-aware and energy-agnostic, depending on whether or not energy efficiency is considered.Energy-aware schemes, such as BiTE, take energy consumption factors into account simultaneously when making load balancing decisions, attempting to strike a balance between performance and energy efficiency. Energy-agnostic schemes, such as DMLBP, focus on optimizing network performance metrics. Traditional schemes often have advantages such as high computational efficiency, strong interpretability, and simple deployment, but may not be as good as machine learning schemes in terms of adaptability and handling complex scenarios.

In choosing data center load balancing schemes, traditional approaches offer simplicity, predictability, and low resource consumption, and are ideal for relatively stable environments with predictable traffic patterns. On the other hand, machine learning-based schemes demonstrate clear advantages in handling complex and dynamic environments, especially when traffic patterns are difficult to describe using simple rules. Among machine learning approaches, supervised learning relies on high-quality historical data but offers stable training, while reinforcement learning optimizes decision-making through continuous interaction at the cost of higher computational overhead. Hybrid schemes like BULB achieve a good balance between performance and efficiency by combining offline training with lightweight online inference.

Incorporating energy awareness adds additional complexity to scheme selection. Although energy optimization can lead to reduced operational costs, it also increases system complexity and computational overhead. One effective solution is to adopt a hierarchical design that handles energy optimization and load balancing relatively independently, balancing various requirements through optimization strategies at different time scales. While this method may not achieve optimality

in all dimensions, it can strike a practical balance among performance, efficiency, and energy consumption, making it particularly suitable for the practical needs of large-scale data centers.

Several innovative schemes integrate multiple technical approaches. HMMLB combines traditional statistical methods with machine learning techniques, using real data from Hermes for training. This integrated approach achieves both the interpretability of traditional algorithms and the adaptive capability of machine learning. BULB employs imitation learning, deploying deep reinforcement learning agents offline while using decision trees for online learning to avoid reinforcement learning problems in the network. This combination of reinforcement learning and machine learning presents a promising direction for future design.

Recent trends show a significant shift from traditional schemes towards machine learning approaches, with this gap continuing to widen. Within machine learning applications, reinforcement learning implementations notably outnumber supervised learning approaches. This trend reflects several factors: the challenges in obtaining high-quality labeled datasets, increasing network dynamics, and recent advances in reinforcement learning techniques.

The evolution of centralized load balancing schemes points towards increased integration with intelligent algorithms, moving beyond traditional approaches and supervised learning towards reinforcement learning and more sophisticated designs. Key challenges in developing centralized load balancing schemes include optimizing reinforcement learning implementations for lightweight deployment while addressing convergence speed and computational resource requirements.

4. Distributed schemes

Distributed schemes operate directly on the data plane, such as on hosts or switches, enabling rapid load balancing decisions without control plane involvement. This architecture allows more fine-grained traffic scheduling. While maintaining global congestion information remains challenging, distributed schemes offer good scalability. The choice of granularity represents a key consideration in distributed schemes, significantly impacting both load balancing effectiveness and overhead. Different granularities introduce distinct challenges. Based on scheduling granularity, distributed load balancing approaches fall into two categories: single-granularity methods and hybrid-granularity methods. The following sections examine recent distributed schemes from these categories, followed by a comprehensive analysis. A summary of distributed approaches is presented in Table 4.

4.1. Single-granularity schemes

In distributed load balancing schemes, selecting an appropriate scheduling granularity represents a critical design choice, driven by different motivations and network environments. Single-granularity schemes simplify design and implementation by performing load balancing at a specific granularity level. This section examines four main single granularity schemes: flow based schemes, flowlet based schemes, flowcell based schemes, and packet based schemes. Based on deployment locations, these schemes are classified into distinct categories. Each granularity has specific application scenarios with unique advantages and limitations. The analysis explores how these schemes balance performance and overhead in different network environments to achieve load balancing.

4.1.1. Flow-based schemes

4.1.1.1. In-network schemes. OPLB [131] is an online elephant flow prediction load balancing system for programmable switch-based data center networks. It adopts a two-stage prediction method: first pre-classifying flows based on stateless features of the first packet, then predicting the size of pre-classified elephant flows using features from the first n packets. Based on the prediction results, elephant flows and mice flows are scheduled to high-throughput and low-latency paths respectively. OPLB leverages the computing and storage capabilities of programmable switches to achieve online updates of models and parameters by mapping trained classification and prediction decision tree models to the match-action pipeline of switches. Experimental results show that OPLB achieves good performance in classification and prediction accuracy, and reduces the amount of information collected by switches.

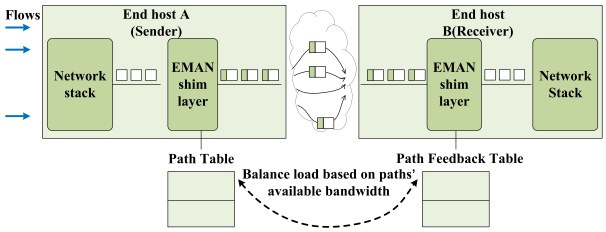


Fig. 17. EMAN structure.

EMAN [132] is an edge-based load balancing scheme for leaf-spine data center networks that requires no modifications to existing network hardware, instead implementing load balancing on end hosts. It monitors the available bandwidth of each path and dynamically allocates traffic based on bandwidth ratios to achieve optimal load balancing. EMAN uses a novel approach of distributing traffic proportionally to each path's available bandwidth, which is detected as the current transmission rate when receiving ECN signals. This allows it to quickly react to link speed degradation and flow competition. Simulation results show that EMAN outperforms existing schemes in both throughput and flow completion time, improving FCT by up to 66% compared to the latest edge-based congestion-aware schemes. The main innovation lies in leveraging edge hosts to achieve distributed load balancing while maintaining low implementation complexity. EMAN is implemented as a hot-pluggable Linux kernel module, transparent to existing applications and kernel TCP stack (see Fig. 17).

CAFT [133] is a distributed congestion-aware fault-tolerant load balancing protocol for three-tier Clos data center networks. It collects complete congestion information of two subsets during the TCP connection process, then uses this information for path selection. CAFT improves robustness to link failures by introducing alternative paths. Large-scale NS-3 simulation results show its advantages in average flow completion time and network throughput, applicable to both symmetric and asymmetric scenarios. The main innovation is proposing a new path selection mechanism that can effectively handle link failures.

DFFR [134] is a distributed flow-level load balancing algorithm for data center networks. It aims to maximize network resource utilization and is scalable, distributed, and adaptive. Through theoretical analysis, the authors prove that the algorithm has verified performance bounds, achieving low variance in aggregate bandwidth utilization. Simulation studies show its superiority over static routing assignment protocols and another distributed algorithm, DDFS, in certain traffic patterns. The main innovation is proposing a new flow-level load balancing algorithm and proving its performance bounds through theoretical analysis.

DDMP [139] is a dynamic distributed multi-path load balancing algorithm designed for hybrid optical/electrical data center networks. Its core innovation lies in the use of dynamic hash computing for network flow distribution, which allows for real-time adjustments of traffic distribution at the microsecond level. The algorithm operates by periodically updating the KEY space distribution based on the inverse ratio of buffer occupancy in top-of-rack (TOR) switches. This approach is particularly effective in addressing the unique challenges of hybrid networks, such as the lack of optical buffers in fast optical switches (FOSs) and the large-grained nature of optical packets. DDMP implements a time slot-based dynamic load balancing scheme, achieving

Table 4
Comparison of distributed load balancing schemes.

Scheme	Category	Performance	Limitations	Suggested improvements
MA-TCP [135]	Flow	Better FCT in both symmetric/asymmetric topologies	Poor short flow performance and limited path visibility	Enhance path visibility and optimize short flow handling
QLLB [136]	Flow	Better short flow performance and high load handling	Continuous monitoring overhead and switch burden	Optimize monitoring and reduce system overhead
BLEND [137]	Flow	40% lower latency and 2.8x throughput for large flows	Controller bottleneck and complex hybrid architecture	Improve controller efficiency and simplify design
OPLB [131]	Flow	Efficient classification with reduced switch collection	Complex prediction process and feature dependencies	Optimize prediction and expand feature set
EMAN [132]	Flow	66% FCT improvement with quick link adaptation	Limited topology support and endpoint complexity	Extend topology support and simplify endpoints
CAFT [133]	Flow	Enhanced FCT and throughput in various scenarios	Limited network support and congestion info needs	Broaden topology support and improve info collection
DFFR [134]	Flow	Guaranteed performance with low utilization variance	Limited adaptability and high computation needs	Enhance adaptability and reduce complexity
DLB [138]	Flow	Better FCT with continuous path migration	TCP modifications and high migration costs	Minimize TCP changes and optimize migration
DDMP [139]	Flow	Reduced latency and loss rate with microsecond scheduling	Network type limitations and frequent updates	Optimize updates and broaden network support
MDTLB [140]	Flow	Good FCT and throughput with path evaluation	Complex classification and parameter adjustment	Simplify classification and parameter tuning
HPLB [141]	Flowlet	Reduced FCT with asymmetric topology support	INT overhead and path selection instability	Reduce overhead and improve path selection
CAF [142]	Flowlet	86% lower FCT and 49.2% higher throughput	Probing overhead and measurement accuracy issues	Enhance measurement accuracy and efficiency
Flex [143]	Flowlet	High adaptability with burst handling	Header modifications and monitoring overhead	Simplify monitoring and state maintenance
MLAB [144]	Flowlet	Low feedback delay with theoretical guarantees	Complex architecture and virtualization needs	Optimize modules and reduce system overhead
Luopan [43]	Flowcell	Efficient handling of long-tail traffic	Switch modifications and sampling issues	Improve sampling and reduce switch changes
RSLB [44]	Flowcell	Better FCT and throughput in asymmetric cases	Complex routing and limited topology support	Simplify routing and extend topology support
QDAPS [58]	Packet	30%–50% lower FCT with flexible balancing	P4 switch needs and estimation overhead	Optimize delay estimation and flow handling
RMC [57]	Packet	72% lower tail FCT with minimal overhead	Hardware modifications and encoding delays	Reduce modifications and encoding overhead
QALL [145]	Packet	Improved FCT and load sharing precision	Complex computation and forwarding instability	Simplify metrics and enhance stability
APS [146]	Packet	60% lower FCT for short flows	Flow classification and reordering issues	Optimize classification and reordering control
LBPP [147]	Packet	Good FCT with burst handling capability	High training overhead and prediction dependency	Reduce training costs and add backup methods
FAMG [148]	Multi	Lower average FCT with good mice flow handling	Endpoint monitoring overhead and suboptimal port selection	Optimize monitoring and improve port selection
FlowDecider [149]	Multi	80% lower short flow FCT, 50% higher throughput	Host involvement and Flowcell table maintenance	Reduce host overhead and enhance port selection
IntFlow [150]	Multi	32% improvement vs. CONGA with active rerouting	Header modification and programmable switch needs	Simplify headers and reduce hardware requirements
BOBBLE [151]	Multi	Improved small flow FCT with dual-mode forwarding	Flow size detection and table maintenance overhead	Optimize flow detection and table management
ILB [152]	Multi	30% lower average FCT, 70% lower tail FCT	Flow conflict detection and frequent rerouting issues	Improve conflict detection and routing stability
AG [153]	Adaptive	Better reordering and link utilization	TCP duplicate ACK dependency and delay measurement	Enhance delay measurement and granularity adjustment
TLB [154]	Adaptive	Low short flow latency with high throughput	Complex load estimation and granularity computation	Simplify load estimation and fragmentation
PDLB [155]	Adaptive	8%–53% FCT improvement with low deployment cost	RTT measurement and optimization model complexity	Improve RTT measurement and model simplification
HG [156]	Adaptive	Timely switching with path-aware adjustment	Complex threshold setting and weight computation	Optimize threshold mechanism and path evaluation
ConWeave [157]	Adaptive	Lower FCT with data plane reordering	Programmable switch and hardware resource needs	Optimize resource usage and reordering mechanism
LBT [158]	Adaptive	55.9–65.4% lower short flow FCT	Complex threshold computation and flow classification	Simplify thresholds and improve classification

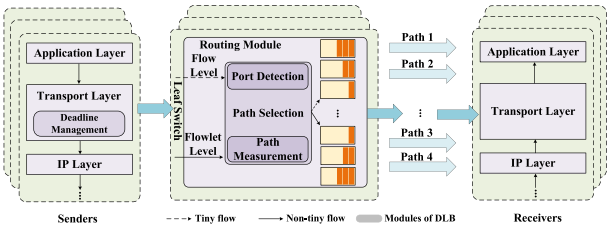


Fig. 18. DLB structure.

microsecond-level traffic grooming for leaf-spine hybrid optical/electrical data center networks. Simulation results demonstrate that DDMP can reduce TOR-to-TOR latency by 15.88% and packet loss by 22.06% compared to conventional algorithms, highlighting its effectiveness in improving network performance.

4.1.1.2. Host-based schemes. MA-TCP [135] is an end-host load balancing approach that innovatively addresses the packet reordering problem in traditional TCP during path migration by sensing TCP flow path migration. Specifically, MA-TCP distinguishes between packet reordering and normal packet loss by recording a Path Migration Point (PMP) in the TCP instance. When reordering is detected, the system does not incorrectly reduce the congestion window but directly triggers fast recovery, thus avoiding unnecessary transmission rate reduction. This local information-based decision mechanism enables MA-TCP to achieve timely path migration and supports continuous migration when congestion occurs on the new path without additional waiting time. Implemented purely in software at the end-host protocol stack, MA-TCP not only avoids the need for network device modifications but also demonstrates excellent performance in both symmetric and asymmetric topologies, particularly in handling large flows and improving FCT.

QLLB [136] is a queue length-based load balancing scheme. It employs a new queue length-based congestion signal that can accurately measure congestion along the path. Additionally, QLLB uses relative RTT to detect asymmetries in the topology. QLLB was implemented in NS-3 and compared with ECMP, CONGA, and Hermes. Results show that QLLB significantly improves the performance of short flows and performs within an acceptable range for long flows. Furthermore, QLLB effectively detects asymmetric paths and outperforms Hermes under high loads. The main innovation of QLLB lies in introducing the queue length-based congestion signal and using relative RTT to detect topology asymmetry, thereby achieving more precise load balancing decisions.

4.1.1.3. Hybrid deployment schemes. MDTLB [140] is a multi-level dynamic traffic load balancing protocol for data center networks. It introduces a five-level path characterization scheme: very good, good, gray, bad, very bad, enabling precise path quality assessment. The protocol employs static and dynamic RTT thresholds for adaptive path judging, enhancing responsiveness to network dynamics. MDTLB's core components include a judging algorithm for path classification and a rerouting algorithm for flow distribution. The scheme adapts parameters based on network load, minimizing congestion mismatch. Evaluated through NS-3 simulations on a leaf-spine topology, MDTLB demonstrated superior flow completion time and throughput across various workloads compared to protocols like ECMP, Clove, and Hermes (see Fig. 19).

BLEND [137] is an SDN-based hybrid load balancing strategy that combines centralized and distributed approaches. It uses a controller to assign paths for elephant flows and calculate globally optimal network queueing delay thresholds, while hosts make local routing decisions based on received path delays. BLEND adopts different strategies for flows of different scales, scheduling elephant flows meticulously to improve throughput and using distributed methods for mice flows to reduce latency. BLEND is easily deployable in data centers and requires no changes to switch hardware. Simulation results show its superiority

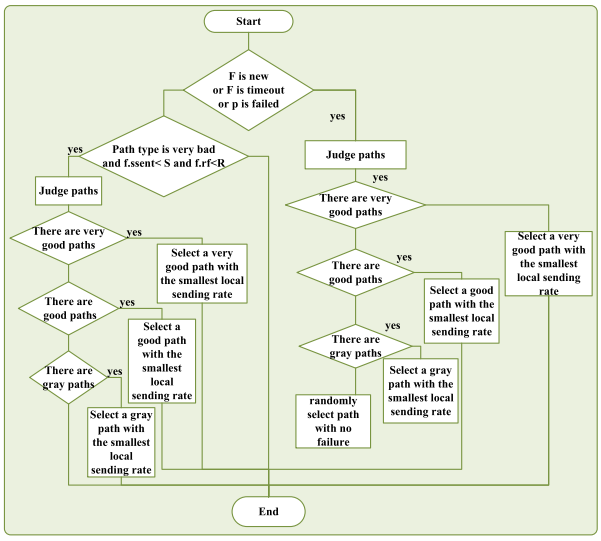


Fig. 19. The flow diagram of MDTLB.

in flow completion time and throughput, reducing tail latency for small and medium flows by up to 40% and improving big flow throughput by up to 2.8 times compared to ECMP. The main innovation is proposing a hybrid architecture that combines centralized and distributed methods, promoting cooperation among network components.

DLB [138] is a multipath-aware TCP scheme for traffic load balancing in data center networks. It can sense the path migration of TCP flows, avoiding the reduction of TCP congestion window caused by packet reordering during path migration. This enables more timely path migration when the original path is congested. Moreover, if the new path becomes congested again, the flow can safely continue to migrate without worrying about transmission rate reduction. NS-3 simulation results show that DLB outperforms existing data center load balancing schemes in terms of flow completion time. The main innovation of this scheme lies in its multipath-aware TCP design, which better adapts to dynamic path changes in data center networks (see Fig. 18).

4.1.2. Flowlet-based schemes

4.1.2.1. In-network schemes. HPLB [141] utilizes in-band network telemetry (INT) to obtain fine-grained path load information, including utilization, timestamp, and queue length. It collects data at non-edge switches and records it in ACK packets, enabling edge switches to make informed routing decisions. HPLB employs a flowlet-based rerouting mechanism, where path selection is probabilistically based on load differences. It uses a water-filling procedure to select available paths with below-average loads. The INT header adds only 31.5 bytes overhead for a typical 5-hop path, representing 2.1% of a 1.5 KB packet. HPLB significantly reduces flow completion time and improves link utilization compared to existing methods like ECMP, CONGA, RPS, and DRILL, especially under asymmetric topologies and realistic data center workloads.

MLAB [144] introduces a modular approach for large-scale, multi-tier DCNs, addressing scalability challenges in modern data centers. It features a fast congestion feedback mechanism, reducing feedback delay to less than a single-trip time between switches. MLAB's modular architecture divides the network into independent routing domains, enhancing deployment flexibility and simplifying network upgrades. Its global load-based routing algorithm allows access switches to select optimal uplink ports, theoretically achieving perfect load balancing across paths. MLAB leverages existing network virtualization technologies for seamless integration, enabling efficient packet encapsulation and decapsulation at network edges. This design ensures compatibility with

current data center infrastructures while providing advanced load balancing capabilities, potentially improving overall network performance and resource utilization in complex, multipath environments.

4.1.2.2. Host-based schemes. CAF [142] addresses congestion mismatch in flowlet-based load balancing schemes through active probing. It comprises flowlet detection, bandwidth measurement, probe gap measurement, RTT estimation, and congestion window adjustment. When a new flowlet is detected, the sender measures available bandwidth using probe packets within one RTT, then sets an appropriate congestion window based on the results. NS-2 simulations and testbed experiments with Web search and data mining workloads in leaf-spine topologies showed CAF outperforms existing schemes, reducing average flow completion time by up to 86% and improving throughput by up to 49.2% in asymmetric scenarios.

Flex [143] is an adaptive flowlet-level load balancing scheme designed to overcome the limitations of fixed timeout-based methods under varying data center network loads. Implemented at the host, it dynamically adjusts flowlet timeout values by continuously monitoring path delays. Flex utilizes TCP timestamp fields to obtain delay information, maintains a FlowInfo table to record flow states, and uses a 1-bit flag in the TCP header's reserved field as a flowlet marker. At the switch level, Flex performs ECMP based on the five-tuple and flowlet marker. NS-3 simulation results show that Flex performs comparably to LetFlow in baseline topologies but excels in handling asymmetric topologies and bursty traffic, particularly reducing flow completion times significantly under Web search workloads.

4.1.3. Flowcell-based schemes

4.1.3.1. In-network schemes. Luopan [43] presents a sampling-based load balancing approach for large-scale data center networks, utilizing flowcells as the granularity unit. Its core mechanism employs periodic sampling where ToR switches send probe packets to destination ToRs, collecting congestion information from random paths using queue length metrics. The switches maintain local congestion tables for path selection, with entries expiring periodically to ensure information freshness. Source routing is implemented by encoding path information in packet headers. While Luopan demonstrates superior scalability and performance in large-scale networks compared to fully global approaches, it requires hardware modifications to commodity switches and may face challenges in adapting to rapid network dynamics due to its static sampling periods.

RSLB [44] introduces a load balancing protocol for software-defined data center networks, employing 64 KB flowcells as transmission units and link delay as the congestion metric. Its key innovation lies in a distributed control structure that utilizes multiple controllers for network-wide congestion monitoring. RSLB implements a three-step routing strategy: selecting optimal links within the source pod, between aggregation switches, and within the destination pod. To enhance scalability, it monitors only uplinks connected to ToR and aggregation-layer switches while leveraging Fat-tree topology symmetry. Experimental results demonstrate RSLB's superior performance in both normal and asymmetric topologies compared to ECMP, Presto, and Hedera, particularly in flow completion time and throughput optimization.

4.1.4. Packet-based schemes

4.1.4.1. In-network schemes. QDAPS [58][1] introduces a queue delay-aware packet spraying approach to mitigate packet reordering issues in asymmetric networks. It selects paths based on output buffer queuing delays, ensuring in-order packet forwarding. The system estimates queuing delays for new packets, records the last packet's delay for each flow, and implements a reordering approach by adjusting forwarding based on comparative delays. For large flows causing long queues, it employs a threshold-based rerouting mechanism. QDAPS achieves high link utilization and reduces flow completion time by 30%–50% compared to state-of-the-art load balancing mechanisms. It operates at the packet level, providing greater flexibility than flow-level or flowlet-level approaches. The system is implemented on switches using P4 language, requiring no modifications to end-host TCP/IP stacks. This implementation allows QDAPS to effectively balance network loads while minimizing packet reordering, resulting in improved overall network performance in data center environments.

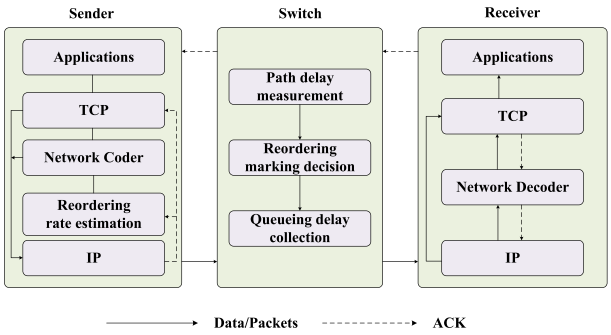


Fig. 20. RMC structure.

QALL [145] leverages programmable data planes to implement fine-grained load balancing based on multiple congestion metrics. It uses local queue behavior measurements for probabilistic port selection on a per-packet/flowlet basis without coordination between switches or controllers. QALL considers queue evolution in load balancing decisions and employs a probabilistic forwarding strategy to evenly distribute traffic across available ports. It integrates various queue behavior metrics to calculate a comprehensive congestion index for each output port, guiding the probabilistic forwarding process. Experimental results under actual DCN workloads demonstrate QALL's superior performance compared to existing schemes in terms of lower flow completion time, shorter decision delay, and smaller load sharing accuracy. QALL's data-driven approach further improves its performance by using least-squares polynomial regression to fit the relationship between egress traffic, queue occupancy, and dequeue time interval. Additionally, QALL's design addresses practical constraints of programmable switches and offers low system overhead, making it suitable for deployment in real-world data center networks.

APS [146] presents an adaptive load balancing approach that addresses resource contention between short and long flows in data center networks. The system employs packet-level load balancing for short flows and ECMP-based flow-level balancing for long flows, effectively isolating these traffic types. Through dynamic path allocation based on short flow presence, APS reduces short flow completion time by up to 60% while increasing long flow throughput by 1.68 times compared to existing approaches. Notably, APS achieves this performance improvement without requiring end-host modifications, facilitating practical deployment.

4.1.4.2. Hybrid deployment schemes. LBPP [147] introduces a hybrid load balancing framework that combines centralized prediction with distributed decision-making for data center networks. At its core, a GNN model deployed on the SDN controller predicts end-to-end delays using an improved Message Passing Neural Network to learn spatiotemporal link correlations. While the controller provides these global predictions, programmable switches make real-time routing decisions locally, enabling both global optimization and low-latency scalability. Evaluations on an 8-pod fat-tree topology demonstrate LBPP's effectiveness in reducing flow completion times, particularly under asymmetric topologies and bursty traffic conditions.

RMC [57] addresses packet reordering challenges in asymmetric data center networks through a dual-component approach: switch-based packet marking and end-system network coding. The system

[1] https://github.com/wenjun95/QDAPS-NS2.

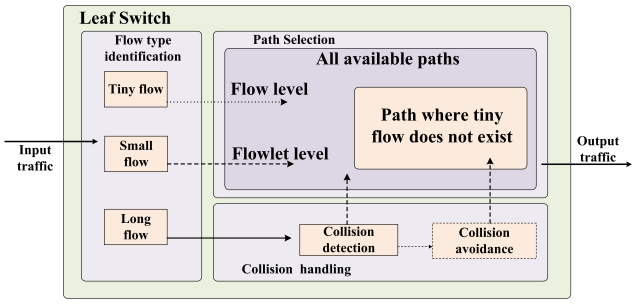


Fig. 21. ILB structure.

proactively identifies out-of-order packets using local queue length and global path delays, marking them via a TCP header bit. At end-systems, RMC implements systematic random linear network coding (RLNC), generating redundant encoded packets based on reordering rates. The scheme dynamically optimizes redundancy to balance decoding probability and queuing delays. Despite introducing minimal overhead (CPU and memory increases of 5% and 0.4% at switches, 0.35% at end-hosts), RMC significantly reduces tail flow completion times by up to 72% through improved packet recovery mechanisms (see Fig. 20).

4.2. Mixed-granularity

As data center network traffic patterns become increasingly complex, single-granularity load balancing schemes face limitations in meeting diverse requirements. Mixed-granularity schemes combine the advantages of different granularities to address diverse network traffic, creating more flexible load balancing schemes.

Mixed-granularity approaches comprise two main categories: multi-granularity schemes and adaptive-granularity schemes. Multi-granularity schemes use multiple granularities simultaneously to handle different types of traffic, while adaptive-granularity schemes dynamically adjust granularity size according to network conditions and environmental changes. These schemes are further classified by their deployment locations to provide detailed discussion.

4.2.1. Multi-granularity schemes

4.2.1.1. In-network schemes. BOBBLE [151] distinguishes between large and small flows, applying packet-level forwarding for short flows and flowlet-level for long flows. It determines flow size by checking packet five-tuples at adjacent time points. BOBBLE implements two routing modes for small flows: random packet spraying and a refilling mode that actively forwards packets to less congested ports. This hybrid approach aims to achieve a balance between per-packet and per-flowlet granularity, leveraging the advantages of both methods. BOBBLE uses a distributed elephant flow detection mechanism and maintains a flowlet table for efficient routing. The refilling mode utilizes a weighted randomization algorithm to select less congested ports, optimizing queue lengths across the network. Experimental results show that BOBBLE significantly improves the flow completion time for small flows in both symmetric and asymmetric topologies, outperforming traditional methods like RPS and LetFlow, while only slightly sacrificing the performance of large flows.

FAMG [148] is a novel data center load balancing scheme that combines per-packet monitoring at hosts with flowlet switching in the network. It can proactively reroute flows based on flow status while avoiding flowlet switching for small flows. FAMG detects large flows at end hosts and splits them into flowcells to pave the way for load balancing. For each flowcell, the switch selects the port with the shortest queue length among two random available ports and the last used port. Small flows are forwarded at the packet level. FAMG has good timeliness and requires no additional hardware support. The scheme introduces an ESF-Approximation Level (EAL) metric to evaluate load balancing effectiveness and employs a mouse flow refilling technique to optimize performance. FAMG demonstrates robustness in both symmetric and asymmetric network topologies. Simulation results show that FAMG significantly reduces average flow completion time while achieving high throughput, particularly for mouse flows in various network conditions.

ILB [152] is an isolation-based load balancing scheme designed to avoid collisions between mixed heterogeneous flows in data center networks. Based on flow type identification, it perceives flow collisions and dynamically assigns different paths to long and short flows. When tiny flows collide with long flows on the same path, ILB immediately forces the latter to reroute to other paths to help the former complete quickly. When tiny flows disappear, long flows quickly occupy all available paths to achieve high throughput. ILB's performance was evaluated through large-scale NS-3 simulation tests under various data center workloads. Experimental results show that while maintaining high throughput for long flows, ILB reduces average and tail flow completion times by 30% and 70% respectively, compared to state-of-the-art load balancing schemes (see Fig. 21).

4.2.1.2. Hybrid deployment schemes. FlowDecider [149] is a proactive load balancing scheme driven by end hosts for data center networks. It detects large flows at the host side and splits them into flowcells in preparation for load balancing. For each flowcell, the switch compares the queue lengths of two random available ports plus the last used port, choosing the smallest one. Small flows are forwarded at the packet level. FlowDecider requires no additional hardware support and has good scalability. The scheme uses a flowcell forwarding table at the switch to efficiently manage flow routing. NS-3 simulation results show that compared to schemes like ECMP and Letflow, FlowDecider reduces the average completion time of short flows by about 80% and 30%, and long flows by about 70% and 30%. Meanwhile, FlowDecider also improves throughput by about 50% and 22%.

IntFlow [150] is an innovative load balancing scheme that integrates packet-based and flowlet-based switching strategies. It evaluates flow status at end hosts to assist flowlet switching in programmable switches. IntFlow uses events such as retransmissions and timeouts, as well as flow sending rate, remaining flow size, and accurate delay of the current path to judge flow status. The sender marks the congestion status of flows in specific fields of the encapsulation header. When flows pass through switches, IntFlow makes careful rerouting decisions based on the flow's congestion marking and flowlet gaps. In this way, IntFlow can proactively reroute flows when congestion or failures are detected, compensating for the shortcomings of traditional flowlet-based schemes. Large-scale NS-3 simulation evaluations show that under asymmetric topologies, IntFlow improves performance by 32% and 28% compared to CONGA and Hermes, respectively.

4.2.2. Adaptive-granularity schemes

4.2.2.1. In-network schemes. AG [153] first provide a detailed analysis of the topology asymmetry problem under existing load balancing mechanisms, including the reordering issue caused by fine-grained load balancing and the low link utilization issue induced by coarse-grained load balancing. To address these problems, AG is proposed, which can dynamically adjust the switching granularity of packets according to the degree of asymmetry among multiple paths. In highly asymmetric topologies, AG increases the granularity to mitigate the reordering problem; in relatively symmetric topologies, AG reduces the granularity to achieve higher link utilization. To obtain topology information, a switch-based scheme is designed, which leverages TCP duplicate ACKs to measure the one-way delay difference between paths and infer the degree of topology asymmetry. AG can alleviate packet reordering in asymmetric scenarios and achieve high link utilization in symmetric scenarios without modifying the host-side protocol stack, making it concise and efficient. AG can be regarded as an adaptive, flowlet-like load balancing scheme.

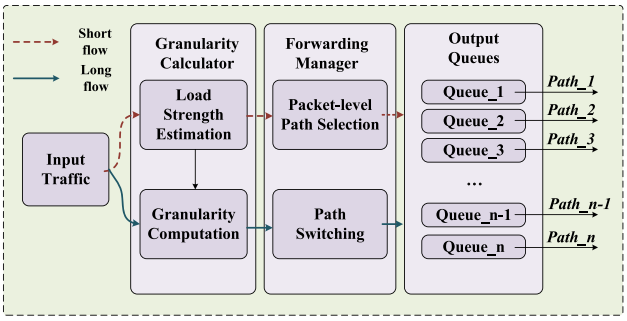


Fig. 22. TLB structure.

TLB [154] is a data center load balancing scheme that prioritizes low-latency for short flows while maintaining high throughput for long flows. It dynamically adjusts the forwarding granularity of long flows based on short flow load intensity. When short flow load is heavy, TLB increases long flow granularity to free up links for short flows. Conversely, when short flow load is light, TLB decreases long flow granularity to improve link utilization. TLB comprises a granularity calculator and a forwarding manager. The calculator estimates short flow load and determines optimal long flow granularity. The manager forwards long flows using this granularity and short flow packets to the shortest queue. For extreme bursts, TLB uses a packet slicing mechanism to yield more bandwidth to short flows (see Fig. 22).

HG [156] improves transmission performance of heterogeneous data center traffic from cloud-based industrial applications. It addresses two defects of existing flowlet-based schemes: untimely switching and disregard for path conditions. HG uses a hybrid-granularity switching strategy, allowing path switches when data volume exceeds a threshold, even before flowlets appear. It divides flows into three phases based on transmitted volume. HG employs a dynamic linear weighting model using packet inter-arrival time and queueing delay to evaluate path status. The model adjusts weights according to flow phase. When switching, HG selects the best path using this model, choosing between the two best-scoring paths with a significant difference.

ConWeave [157][2] is a load balancing framework specifically designed for RDMA networks [159], addressing issues related to RDMA flow characteristics and out-of-order packet handling. It comprises a source ToR that performs cautious rerouting and a destination ToR that reorders out-of-order packets in the data plane using programmable switch capabilities. Notably, ConWeave uses RTT monitoring to implement adaptive-granularity rerouting, allowing it to make frequent rerouting decisions based on network measurements. The design motivation stems from the limitations of existing load balancing schemes in RDMA networks: flowlet-based methods are ineffective due to the lack of sufficient flowlet gaps, while packet-level methods result in numerous out-of-order packets, triggering unnecessary rate reductions and retransmissions. Evaluations demonstrate that ConWeave significantly reduces average and tail flow completion times, particularly excelling in asymmetric topology and bursty traffic scenarios, while maintaining moderate hardware resource usage.

LBT [158] is a load balancing scheme for heterogeneous traffic. Its key design point is to adaptively adjust the switching granularity of long flows and best-effort flows according to two switching thresholds calculated based on traffic intensity. Specifically, under heavy load, the switching granularity is increased to guarantee the required bandwidth capacity for delay-sensitive short flows to finish quickly. Conversely, under light load, the switching granularity is reduced to enable long flows to fully utilize parallel equal-cost paths. Additionally, LBT adopts different routing strategies for three different categories of flows. NS-2 simulation evaluations show that compared to existing schemes, LBT significantly reduces the average completion time of short flows by up to 55.9%–65.4%, while also achieving high throughput for long flows.

4.2.2.2. Host-based scheme. The core idea of PDLB [155] aims to reduce packet reordering in asymmetric topologies while improving link utilization by adaptively adjusting flowcell granularity. It consists of a path asymmetry estimation module and a flowcell size adjustment module. The estimation module measures RTT of each path to quantify asymmetry. The adjustment module computes optimal flowcell granularity based on asymmetry degree. When asymmetry is high, PDLB increases granularity to reduce reordering; when low, it decreases granularity to improve utilization. PDLB models the relationship between flow completion time and granularity for fast optimization. Implemented only at the sender side, PDLB has low deployment overhead. NS-2 simulations show PDLB improves short flow completion times by 8% to 53% compared to existing schemes.

4.3. Discussion of distributed schemes

This section analyzes distributed load balancing schemes for data center networks, classifying them into two main types: single-granularity and mixed-granularity schemes. Each scheme represents distinct approaches to network performance optimization.

Single-granularity schemes employ different granularities according to specific performance goals. Flow-level schemes like CAFT and DFFR prioritize improving link utilization but risk short-term load imbalances. Packet-level schemes such as QDAPS and RMC address reordering issues, enabling finer-grained load balancing while potentially increasing reordering overhead. Flowlet and flowcell schemes balance link utilization with packet reordering. Although these single-granularity schemes perform well in specific scenarios, they have limitations in meeting the diverse requirements of data center networks.

Mixed-granularity schemes offer a more flexible design approach, addressing different types of flows. These schemes accommodate traffic heterogeneity in data center networks by applying varied granularities for load balancing across different flows. Two main design approaches emerge: multi-granularity schemes and adaptive granularity adjustment. Multi-granularity schemes like FAMG and LBT apply different granularities based on flow types. Adaptive granularity adjustment schemes such as AG and PDLB dynamically modify transmission granularity to respond to network changes. Some innovative schemes like ConWeave introduce new granularity definitions, achieving an effective balance between link utilization and reordering issues.

In terms of specific implementations, different schemes make unique design trade-offs for particular scenarios. HPLB employs fine-grained information collection based on INT, which performs well in handling dynamic loads, but its 2.1% additional overhead may become a bottleneck in large-scale deployments. In contrast, Luopan's sampling-based design provides a more lightweight alternative, sacrificing some accuracy for lower overhead. QALL achieves multi-metric load balancing through programmable data planes, demonstrating excellent performance, but its complex design may face resource constraints. Comparatively, APS's traffic classification scheme, although slightly inferior in performance, is simpler and more practical to implement. Regarding network asymmetry issues, AG adapts by dynamically adjusting granularity, while RMC employs switch marking in conjunction with end-host coding for handling such problems.

Analysis of recent research demonstrates the prevalence of single granularity schemes. Among the schemes examined in this chapter, 21 implement single-granularity methods, while 11 use mixed-granularity approaches. These numbers highlight the balance between implementation simplicity and performance adaptability.

Future research points to parallel development of both single and mixed granularity schemes. Single-granularity schemes remain optimal for small data centers with simple structures and stable traffic patterns,

[2] https://github.com/conweave-project/conweave-ns3.

while mixed-granularity schemes offer advantages for large, complex data center networks. The development of mixed-granularity schemes presents specific challenges, including granularity type selection and adjustment trigger conditions, which significantly impact scheme performance. Beyond granularity selection, scheme performance, scalability, and implementation complexity depend heavily on deployment location (e.g., in-network or host-based) and the utilization of global information.

As data center networks grow in scale and complexity, distributed load balancing schemes require increasingly sophisticated designs. Key areas for future research include optimizing granularity selection for different scenarios, enhancing congestion information utilization, and leveraging emerging technologies such as machine learning and programmable switches to improve load balancing decisions.

5. Summary and future insights

Section 5.1 summarizes the metrics used in the discussed load balancing schemes. Table 5 presents these metrics, illustrating the indicators used by various schemes. This analysis offers insights into the key dimensions of load balancing scheme design, essential for evaluating different schemes' strengths and weaknesses. Section 5.2 evaluates both centralized and distributed schemes through experimental testbeds. Section 5.3 explores future developments and research directions in the field.

5.1. Summary of metrics in load balancing

Performance metrics provide quantitative evaluation of load balancing schemes in data center networks. Different schemes employ distinct evaluation indicators, as summarized below:

Flow Completion Time: FCT refers to the total time from when the first packet of a flow is sent into the network until the last packet of that flow is received and acknowledged as complete. In modern schemes, various derivatives of FCT are often used, such as average FCT, 99th percentile FCT, etc., to measure the performance of the schemes.

Throughput: Throughput quantifies the system's efficiency by measuring the number of successfully completed requests per unit time, as they traverse from the source to the destination. This metric reflects the rate at which a network or system can process and deliver data. Average throughput is often used as a performance indicator, providing a more comprehensive view of the system's performance over time.

Utilization: It represents the ratio of actually used resources to the total available resources, typically expressed as a percentage. In the networking field, there are various specific utilization metrics, such as link utilization, bandwidth utilization, and network utilization. Additionally, there are metrics like CPU utilization and buffer utilization. These indicators provide important reference points for network operations and management.

Latency: It refers to the various time consumptions during the entire process of data transmission from the source node to the destination node in a network. This metric reflects the response speed of the network. In different network segments, various types of specific delays may be encountered, such as communication delay, routing delay, and queuing delay. Lower latency typically indicates faster data transmission and better network performance.

Bandwidth: Bandwidth is a crucial metric in network communications used to measure data transmission capacity. It refers to the maximum amount of data that a network connection can transmit within a given time period (typically one second). Bandwidth is usually measured in bits per second (bps), with common representations including Mbps or Gbps. Higher bandwidth indicates that the network can transmit large amounts of data more quickly.

Overhead: It refers to the additional resources consumed in the process of network communication to achieve specific functions or maintain system operation. These resources may include processing time, scheduling capacity, bandwidth. Overhead is typically viewed as a necessary cost, but excessive overhead can affect the overall efficiency and performance of the system.

Energy Consumption: It refers to the total amount of energy consumed by nodes in the network (such as servers, routers, etc.) when processing various requests. Effective load balancing strategies can help optimize resource utilization, reduce overall energy consumption, thereby lowering operational costs and improving system sustainability.

Reliability: Reliability is the ability of a network system to consistently perform its intended functions correctly under specified conditions and time periods, reflecting the system's capacity to maintain stable operation and avoid failures and errors. It is typically measured through various specific indicators, such as retransmission rate and flow success rate. Lower retransmission rates and higher flow success rates indicate a more reliable network.

Scalability: Scalability refers to the ability of a network system to maintain or improve its performance and efficiency as its scale expands and complexity increases. It reflects the system's flexibility to adapt to growth and increased management demands without significantly degrading performance or increasing management complexity. Scalability is typically measured through multiple indicators, such as running time and the amount of message exchanges.

Packet Loss Rate: It measures the percentage of transmitted packets that fail to reach their destination. It is calculated by comparing unreceived packets to total sent packets. Network efficiency is optimized by maintaining a low packet loss rate. This metric is crucial for assessing network performance and reliability.

Round Trip Time: It refers to the total time it takes for data to travel from the sender to the receiver and back to the sender. This metric is crucial for evaluating network performance because it determines the timing of retransmission strategies. If a system underestimates this time, it may lead to premature data retransmission, potentially initiating new transmissions even while the original acknowledgment is still in transit. This situation can result in network congestion. Conversely, if the estimated value is too high, the system may set excessively long timeout periods. This can cause slow responses to data loss, reducing the overall efficiency of network transmission.

Deadline Miss Rate: It represents the ratio of tasks that fail to complete within their specified deadlines to the total number of tasks during system operation. A lower DMR indicates that the system is better able to meet real-time requirements, while a higher DMR suggests that the system may need optimization or redesign to improve its real-time performance.

Response Time: Response time refers to the time interval between the moment a system receives a request and the moment it begins processing that request. It reflects the system's speed of reaction to input stimuli. Shorter response times typically indicate better system performance and user experience.

Reordering Degree: It refers to a quantitative indicator of the phenomenon where the order of packets received at the destination does not match the order in which they were sent from the source during network transmission. This can be measured in various ways, such as the proportion of out-of-order packets or the severity of the reordering. A lower degree of reordering or reordering rate typically indicates better network performance and transmission quality.

Congestion Window: The congestion window is a key parameter in TCP protocol used for flow control, representing the maximum amount of data a sender can transmit before receiving an acknowledgment from the receiver. It adjusts dynamically to reflect the sender's estimate of network congestion. A larger congestion window typically indicates higher network throughput. In some packet-level schemes, the size of the congestion window is often used as a metric to compare the performance of different approaches, reflecting their adaptability to network conditions and transmission efficiency.

Table 5
Metrics of load balancing in data center networks.

Ref.	Flow completion time	Throughput	Utilization	Latency	Bandwidth	Overhead	Energy consumption	Reliability	Scalability	Packet loss rate	Round trip time	Deadline miss rate	Response time	Reordering degree	Congestion window	Jitter	Degree of LB
HMMLB [83]			✓		✓	✓											
ANN-KDN [85]				✓	✓												
LLB [84]													✓				
Babayigit et al. [87]													✓				
LLBR [82]			✓	✓						✓						✓	
BULB [86]	✓			✓													✓
SPRLN [93]		✓	✓							✓	✓					✓	
Tosounidis et al. [89]	✓	✓						✓									
EDDQN [94]		✓	✓	✓													
GDLB [90]			✓	✓						✓							
DRLB [91]	✓	✓															
TRO [123]		✓									✓						✓
DeepRLB [88]	✓		✓														✓
DRL-TAL [92]	✓	✓		✓										✓			
RWCMP [96]			✓														
RILNET [95]			✓	✓						✓							
DRL-Plink [97]	✓											✓					
SAPS [102]	✓																
AggreFlow [98]						✓	✓			✓							✓
BiTE [99]			✓				✓										
ESLB [100]		✓	✓				✓										
Pathan et al. [101]							✓	✓									✓
PSO [103]		✓	✓	✓													
IALB [104]		✓	✓														
Zheng et al. [105]			✓														
Chung et al. [106]			✓														
Oddlab [107]	✓		✓														
MATCP [135]	✓																
OPLB [131]		✓															
MDTLB [140]	✓	✓															
QLLB [136]	✓																
DLB [138]	✓					✓						✓					
EMAN [132]	✓	✓															
BLEND [137]	✓	✓															
CAFT [133]	✓	✓															
DFFR [134]	✓				✓												
DDMP [139]		✓		✓						✓							
Luopan [43]	✓																
RSLB [44]	✓	✓							✓								
CAF [142]		✓				✓				✓							
HPLB [141]	✓	✓															
MLAB [144]	✓			✓													
QALL [145]	✓			✓													✓
QDAPS [58]	✓		✓											✓	✓		
RMC [57]	✓	✓		✓		✓		✓							✓		
Flex [143]	✓	✓												✓			
APS [146]	✓	✓	✓	✓								✓		✓			
LBPP [147]	✓																
BOBBLE [151]	✓																
FAMG [148]	✓																
FlowDecider [149]	✓	✓															
IntFlow [150]	✓																
LBT [158]	✓	✓															
AG [153]	✓		✓			✓								✓			
HG [156]	✓	✓		✓													
ConWeave [157]	✓																
PDLB [155]	✓		✓											✓			
TLB [154]	✓	✓				✓						✓					
ILB [152]	✓	✓		✓		✓		✓									

Jitter: Jitter in network communications refers to the variability in packet delay between network nodes. It is quantified as the inconsistency in latency when data packets travel from one network element to another. Network congestion tends to exacerbate jitter, leading to increased unpredictability in packet arrival times.

Degree of Load Balancing: The degree of load balancing is an indicator used to measure the uniformity of load distribution in a system. It can be defined and calculated in various ways, such as statistical measures of link utilization or queue length (variance, standard deviation, etc.) , specialized balance indices (like Jain's fairness index), or custom indicators designed for specific needs. For example, the ESF-Approximation Level proposed in FAMG [148] measures the length differences between queues to indicate the degree of load balancing.

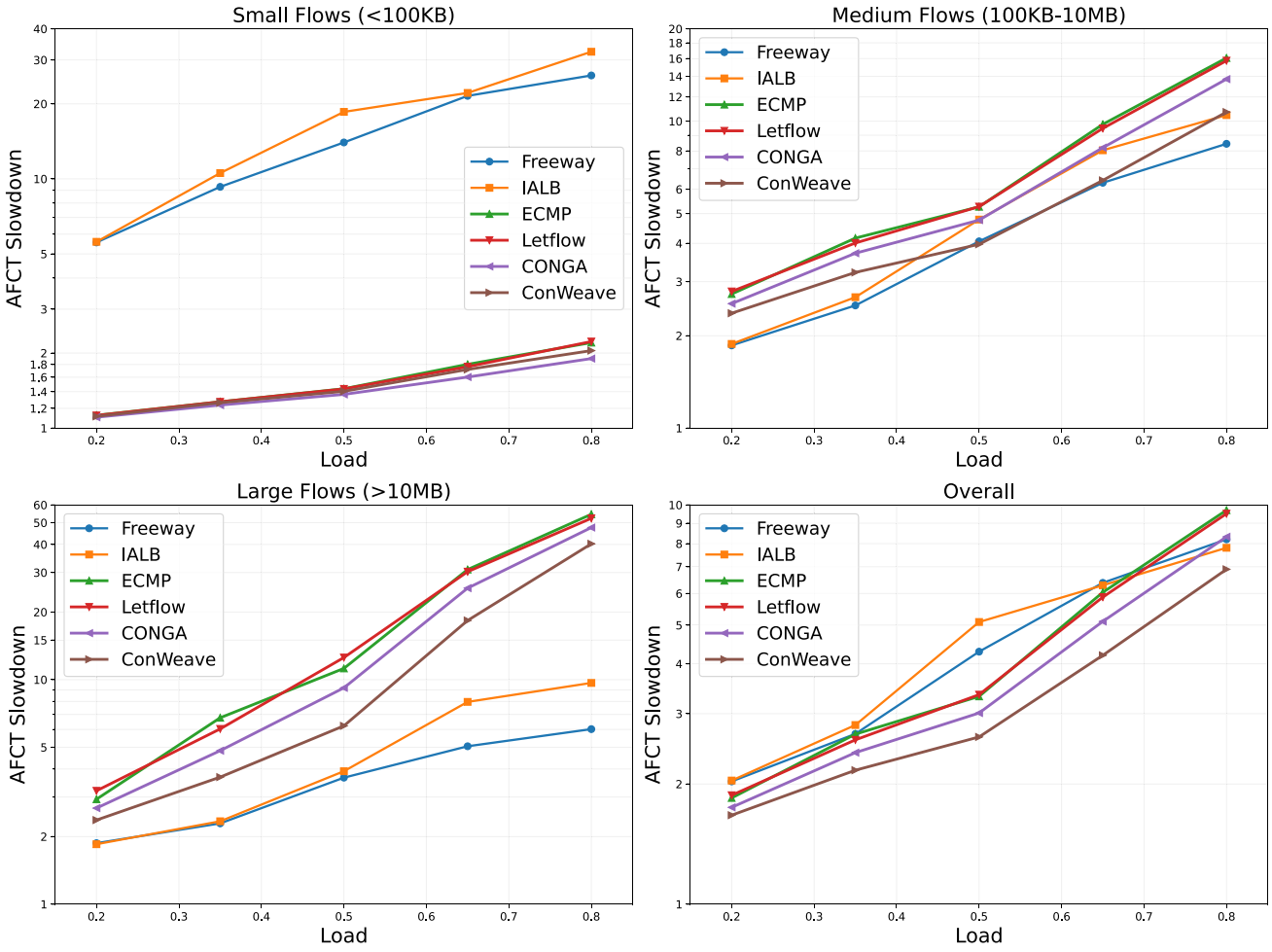


Fig. 23. Experiment results.

5.2. Quantitative analysis of centralized and distributed schemes

Six reproducible schemes were selected for quantitative analysis: ECMP, Conga, LetFlow, Conweave, Freeway and IALB. The re-implementation process maintained each scheme's original platform environment: centralized schemes on a combination of Ryu and Mininet, and distributed schemes on the NS3 platform. Testing utilized a K = 4 Fat-Tree topology and WebSearch workload across both platforms with a unified traffic generator to ensure comparable results. Performance evaluation focused on AFCT Slowdown measurements (see Fig. 23).

The significance of AFCT Slowdown lies in its representation of the ratio between the actual average FCT and the ideal average FCT under a specific load. This metric reflects the relative performance change of a scheme as the load increases.

For small flows (<100 KB), distributed schemes exhibit clear advantages. At a load of 0.2, the FCT Slowdown of distributed schemes is only around 1.1–1.2, while centralized schemes reach 5.5–5.6. This gap widens as the load increases. At a load of 0.8, the Slowdown of distributed schemes is 2.0–2.2, while centralized schemes rise to 26–32. This difference mainly stems from the inherent control overhead of centralized schemes: each flow needs to communicate with the controller to obtain routing decisions, and this overhead is particularly significant for small flows.

Medium-sized flows (100 KB–10 MB) exhibit a different pattern. At low loads (0.2), centralized schemes perform better, with a Slowdown of approximately 1.9, while distributed schemes range between 2.3–2.7. However, as the load increases, the advantages of distributed schemes gradually emerge. At a load of 0.8, the Slowdown of centralized schemes is 8–10, while distributed schemes, although also experiencing performance degradation, perform better overall, especially ConWeave, which maintains a lower level.

Large flows (>10 MB) best reflect the characteristics of the two types of schemes. At low loads, centralized schemes show an advantage, with a Slowdown of only 1.8–1.9, while distributed schemes range between 2.3–3.2. This advantage persists throughout the load increase. Even at a high load of 0.8, the Slowdown of centralized schemes remains between 6–10, while distributed schemes rise to 40–54. This reflects the advantage of centralized schemes in making routing decisions with a global perspective, enabling them to select better transmission paths for large flows.

From an overall average performance perspective, the differences between the two types of schemes exhibit interesting changes under different loads. At low loads, both perform similarly, but as the load increases, distributed schemes show a slight advantage. This indicates that when considering various flow sizes comprehensively, the overall performance of distributed schemes is more stable.

The analysis highlights a critical design trade-off: centralized schemes leverage global perspective for optimal routing decisions but incur significant control overhead for small flows. Distributed schemes, while potentially less optimal for large flows, excel at handling small flows through low-latency and fast-response capabilities. These performance characteristics suggest that deployment decisions should consider traffic patterns, potentially incorporating hybrid architectures that combine the strengths of both approaches.

5.3. Future insights

Based on our previous discussions, this section presents insights into future development trends of data center network load balancing. The analysis encompasses five key aspects: hardware, software, architecture, algorithms, and open challenges.

5.3.1. Load balancing innovation based on emerging network hardware

Hardware advances have transformed the design and deployment of load balancing schemes. Traditional data center network devices served primarily as data forwarders, requiring specialized equipment for complex load balancing strategies. The emergence of programmable switches and smart NICs has extended computing capabilities to network edges, establishing new in-network computing capabilities. Programmable hardware provides unprecedented data plane flexibility beyond simple forwarding rule matching. These advances enable microsecond-level load balancing decisions, including lightweight machine learning and reinforcement learning models in the data plane [160]. In-band network telemetry allows load balancing schemes to better monitor and adapt to network changes, reducing control plane overhead.

RDMA technology has revolutionized data transmission methods. Direct data transmission at the NIC level bypasses portions of the traditional network protocol stack, enabling ultra-low latency load balancing. Smart NICs provide powerful local computing capabilities, supporting complex load balancing algorithms at network edges, such as feature extraction [161].

While these hardware innovations advance load balancing capabilities, they present new challenges. In lossless data center networks, Priority-based Flow Control (PFC) prevents packet loss effectively, but packet reordering remains a significant challenge. Future load balancing schemes must address these emerging issues while leveraging advanced hardware capabilities.

5.3.2. Efficient load balancing based on user-space protocol stacks and NFV

Software advances complement hardware transformation in enhancing load balancing efficiency. Traditional load balancing schemes operate primarily in kernel space, processing packets through the complete Linux network protocol stack. While functionally complete, this approach introduces significant interrupt overhead and processing latency. User-space network protocol stacks offer solutions to these limitations, presenting innovative approaches to data center network load balancing [162,163]. Bypassing the kernel with streamlined protocol stacks in user space reduces overhead and processing latency. Integration with DPDK technology enables ultra-low latency load balancing at the software level. NFV technology further advances load balancing by decoupling network functions from dedicated hardware, enabling flexible software deployment of load balancing services and reducing deployment costs. As data center networks become increasingly specialized, load balancing schemes require deeper integration with specific application scenarios, demanding refined performance optimization and decision-making mechanisms.

5.3.3. Load balancing architecture design for multi-point coordination

Load balancing architecture continues to advance in search of optimal solutions. After years of parallel development, both centralized and distributed schemes now trend towards multi-point coordinated deployment architectures. The evolution from single-point deployment on switches and controllers to coordinated decision-making across switches, controllers, and host endpoints has enhanced overall system performance.

Multi-point deployment enhances load balancing schemes through several key advantages. Distributed algorithms across different network locations reduce decision latency while enabling more precise routing decisions through differentiated control. The architecture leverages diverse information sources: controllers maintain global network views, switches monitor real-time local traffic states, and host endpoints access detailed application-layer information. This multi-dimensional data supports more informed load balancing decisions.

While multi-point deployment architectures effectively address the distributed, heterogeneous nature of modern data center networks, they present specific challenges. These include managing increased system complexity, optimizing communication overhead, and maintaining decision consistency across endpoints. These aspects require further research and development.

5.3.4. Deep integration of machine learning and load balancing strategies

Machine learning has transformed the approach to load balancing decisions. Various machine learning algorithms contribute distinctly to different aspects of load balancing. Both large neural networks deployed on control planes and lightweight algorithms implemented on programmable hardware have significantly influenced load balancing scheme design. The following analysis examines specific functional implementations, matching algorithm types to their optimal applications.

Machine learning algorithms demonstrate clear hierarchical patterns in traffic classification and identification. Traditional machine learning algorithms such as SVM and RF excel at basic tasks like five-tuple-based classification and traffic type identification. These algorithms deliver quick decisions with minimal resource consumption, optimizing performance in resource-constrained networks. For complex application-layer traffic identification and anomaly detection, deep neural networks excel through automatic feature learning and enhanced identification accuracy.

In traffic prediction and load estimation, different machine learning algorithms address distinct prediction cycles and scenarios. RNN-type networks excel at short-term traffic prediction, while Transformer-type models demonstrate superior accuracy in long-term trend prediction [164]. These temporal algorithms remain essential for advanced traffic analysis.

Reinforcement learning has revolutionized direct load decision optimization. The algorithms optimize decision strategies through environmental interaction, enhancing load balancing effectiveness. This approach particularly suits dynamic network environments, offering advantages over supervised learning methods that rely on large datasets.

The integration of machine learning into load balancing systems presents several key challenges: selecting optimal ML algorithms, identifying load balancing components for algorithmic enhancement, integrating algorithms efficiently with existing hardware and software infrastructures, and developing ML approaches compatible with system architectures. Machine learning algorithms are expected to extend across all aspects of load balancing systems, establishing new operational paradigms.

5.3.5. Open issues: Energy and large language models

Data center network load balancing faces emerging challenges amidst evolving technology landscapes. As data centers expand and energy constraints tighten, efficiency optimization remains a critical long-term consideration in load balancing design. Despite fewer energy-focused proposals in recent research, the significance of energy efficiency persists. Emerging technologies and requirements present new opportunities for innovation, particularly in expanding scenarios like edge computing and distributed data centers, where energy-optimized load balancing requires comprehensive approaches. The intersection of load balancing and energy efficiency optimization will likely continue as a key research focus as data centers scale and evolve.

Emerging applications present both challenges and opportunities. The widespread adoption of LLMs across industries requires data centers to adapt their load balancing schemes for these specific traffic patterns. Recent works [165,166] demonstrate the effectiveness of intent-driven approaches in network management and load balancing, translating high-level policies and intents into specific load balancing

strategies. LLMs offer potential enhancements to intent-based load balancing, from implementation and deployment to dynamic strategy adjustment based on network conditions. This evolution advances load balancing systems beyond traditional rule-based approaches towards intelligent, intent-aware solutions.

6. Conclusion

This survey examines the current state of load balancing techniques in modern data center networks, covering centralized and distributed schemes, their design principles, implementation mechanisms, advantages, and limitations. The analysis highlights trade-offs between different approaches under centralized and distributed classifications.

Future research points towards deeper integration of programmable data planes and intelligent algorithms for more precise, adaptive real-time load balancing decisions. Emerging hybrid schemes combine features across traditional boundaries, leveraging strengths while addressing limitations of various approaches. The integration of large models continues to shape the evolution of load balancing schemes.

As data center networks evolve, advances in load balancing techniques remain essential for maintaining the performance, reliability, and scalability of these critical infrastructure components.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] Cisco, Cisco Annual Internet Report (2018–2023) White Paper, Technical report, 2020.

[2] Christian Hopps, Analysis of an Equal-Cost Multi-Path Algorithm, Technical report, 2000.

[3] Mohammad Alizadeh, Tom Edsall, Sarang Dharmapurikar, Ramanan Vaidyanathan, Kevin Chu, Andy Fingerhut, Vinh The Lam, Francis Matus, Rong Pan, Navindra Yadav, et al., CONGA: Distributed congestion-aware load balancing for datacenters, in: Proceedings of the 2014 ACM Conference on SIGCOMM, 2014, pp. 503–514.

[4] Soudeh Ghorbani, Zibin Yang, P Brighten Godfrey, Yashar Ganjali, Amin Firoozshahian, Drill: Micro load balancing for low-latency data center networks, in: Proceedings of the Conference of the ACM Special Interest Group on Data Communication, 2017, pp. 225–238.

[5] Hong Zhang, Junxue Zhang, Wei Bai, Kai Chen, Mosharaf Chowdhury, Resilient datacenter load balancing in the wild, in: Proceedings of the Conference of the ACM Special Interest Group on Data Communication, 2017, pp. 253–266.

[6] Keqiang He, Eric Rozner, Kanak Agarwal, Wes Felter, John Carter, Aditya Akella, Presto: Edge-based load balancing for fast datacenter networks, ACM SIGCOMM Comput. Commun. Rev. 45 (4) (2015) 465–478.

[7] Alex Ramiro Masaquiza Caiza, Denis Andres Maigualema Quimbita, Mauro Tropea, Load balancing algorithms in SDN networks with multiple servers, in: Disruptive Technologies in Information Sciences VIII, Vol. 13058, SPIE, 2024, pp. 424–434.

[8] Mattia Giovanni Spina, Mauro Tropea, Floriano De Rango, SURA-LB: Software-defined IDS with UAV resource aware load-balancing in FANET disaster scenarios, Comput. Commun. 223 (2024) 101–114.

[9] DF Galiaskarov, NN Nikolaychuk, SN Stepanov, AP Pshenichnikov, MG Kanishcheva, Dynamic load balancing in data center, in: 2024 Systems of Signals Generating and Processing in the Field of on Board Communications, IEEE, 2024, pp. 1–7.

[10] Jiao Zhang, F Richard Yu, Shuo Wang, Tao Huang, Zengyi Liu, Yunjie Liu, Load balancing in data center networks: A survey, IEEE Commun. Surv. Tutorials 20 (3) (2018) 2324–2352.

[11] Avnish Thakur, Major Singh Goraya, A taxonomic survey on load balancing in cloud, J. Netw. Comput. Appl. 98 (2017) 43–57.

[12] Einollah Jafarnejad Ghomi, Amir Masoud Rahmani, Nooruldeen Nasih Qader, Load-balancing algorithms in cloud computing: A survey, J. Netw. Comput. Appl. 88 (2017) 50–71.

[13] Majd Mutaher Hasan, Nabel Alsohaibi, A review on load balancing algorithms in cloud computing, 2022.

[14] Juliet G. Muchori, Peter M. Mwangi, Machine learning load balancing techniques in cloud computing: a review, 2022.

[15] Weihe Li, Jingling Liu, Shiqi Wang, Tao Zhang, Shaojun Zou, Jinbin Hu, Wanchun Jiang, Jiawei Huang, Survey on traffic management in data center network: from link layer to application layer, IEEE Access 9 (2021) 38427–38456.

[16] Sura Hacham, Norashidah Md Din, Nagaletchumi Balasubramanian, Load balancing in software-defined data centre with fat tree architecture, in: 2022 4th International Conference on Smart Sensors and Application, ICSSA, IEEE, 2022, pp. 45–49.

[17] Zhenzao Wen, Linghe Kong, Guihai Chen, Muhammad Khurram Khan, Shahid Mumtaz, Joel JPC Rodrigues, Understanding multi-path routing algorithms in datacenter networks, in: 2019 IEEE Global Communications Conference, GLOBECOM, IEEE, 2019, pp. 1–6.

[18] Ting Wang, Bo Li, Mingsong Chen, Shui Yu, Machine learning empowered intelligent data center networking, in: Machine Learning Empowered Intelligent Data Center Networking: Evolution, Challenges and Opportunities, Springer, 2022, pp. 15–99.

[19] Qin Du, Xin Cui, Haoyao Tang, Xiangxiao Chen, Review of load balancing mechanisms in SDN-based data centers, J. Comput. Commun. 12 (01) (2024) 49–66.

[20] Yong Liu, Tianyi Yu, Qian Meng, Quanze Liu, Flow optimization strategies in data center networks: A survey, J. Netw. Comput. Appl. (2024) 103883.

[21] Mohammad Al-Fares, Alexander Loukissas, Amin Vahdat, A scalable, commodity data center network architecture, ACM SIGCOMM Comput. Commun. Rev. 38 (4) (2008) 63–74.

[22] Albert Greenberg, James R Hamilton, Navendu Jain, Srikanth Kandula, Changhoon Kim, Parantap Lahiri, David A Maltz, Parveen Patel, Sudipta Sengupta, VL2: A scalable and flexible data center network, in: Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication, 2009, pp. 51–62.

[23] Chuanxiong Guo, Guohan Lu, Dan Li, Haitao Wu, Xuan Zhang, Yunfeng Shi, Chen Tian, Yongguang Zhang, Songwu Lu, Bcube: a high performance, server-centric network architecture for modular data centers, in: Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication, 2009, pp. 63–74.

[24] Chuanxiong Guo, Haitao Wu, Kun Tan, Lei Shi, Yongguang Zhang, Songwu Lu, Dcell: a scalable and fault-tolerant network structure for data centers, in: Proceedings of the ACM SIGCOMM 2008 Conference on Data Communication, 2008, pp. 75–86.

[25] Radhika Niranjan Mysore, Andreas Pamboris, Nathan Farrington, Nelson Huang, Pardis Miri, Sivasankar Radhakrishnan, Vikram Subramanya, Amin Vahdat, Portland: a scalable fault-tolerant layer 2 data center network fabric, in: Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication, 2009, pp. 39–50.

[26] Ankit Singla, Chi-Yao Hong, Lucian Popa, P Brighten Godfrey, Jellyfish: Networking data centers randomly, in: 9th USENIX Symposium on Networked Systems Design and Implementation, NSDI 12, 2012, pp. 225–238.

[27] Moeen Al-Makhlafi, Huaxi Gu, Ahlam Almuaalemi, Eiad Almekhlafi, Musbahu M Adam, RibsNet: A scalable, high-performance, and cost-effective two-layer-based cloud data center network architecture, IEEE Trans. Netw. Serv. Manag. 20 (2) (2022) 1676–1690.

[28] Zina Chkirbene, Rachid Hadjidj, Sebti Foufou, Ridha Hamila, LaScaDa: A novel scalable topology for data center network, IEEE/ACM Trans. Netw. 28 (5) (2020) 2051–2064.

[29] Deshun Li, Yanming Shen, Keqiu Li, FleCube: A flexibly-connected architecture of data center networks on multi-port servers, Comput. Commun. 77 (2016) 62–71.

[30] Dan Li, Chuanxiong Guo, Haitao Wu, Kun Tan, Yongguang Zhang, Songwu Lu, Ficonn: Using backup port for server interconnection in data centers, in: IEEE INFOCOM 2009, IEEE, 2009, pp. 2276–2285.

[31] Da ming Yu, Zhen Zhang, Yu hui Deng, Long xin Lin, Teng jiao He, Guang liang He, Flexible, highly scalable and cost-effective network structures for data centers, J. Netw. Comput. Appl. (ISSN: 1084-8045) 210 (2023) 103542, http://dx.doi.org/10.1016/j.jnca.2022.103542, URL https://www.sciencedirect.com/science/article/pii/S1084804522001837.

[32] Nathan Farrington, George Porter, Sivasankar Radhakrishnan, Hamid Hajabdolali Bazzaz, Vikram Subramanya, Yeshaiahu Fainman, George Papen, Amin Vahdat, Helios: a hybrid electrical/optical switch architecture for modular data centers, in: Proceedings of the ACM SIGCOMM 2010 Conference, 2010, pp. 339–350.

[33] Hao Feng, Yuhui Deng, Xiao Qin, Geyong Min, Criso: An incremental scalable and cost-effective network architecture for data centers, IEEE Trans. Netw. Serv. Manag. 18 (2) (2020) 2016–2029.

[34] Haitao Wu, Guohan Lu, Dan Li, Chuanxiong Guo, Yongguang Zhang, Mdcube: a high performance network structure for modular data center interconnection, in: Proceedings of the 5th International Conference on Emerging Networking Experiments and Technologies, 2009, pp. 25–36.

[35] Guijuan Wang, Yazhi Zhang, Jiguo Yu, Meijie Ma, Chunqiang Hu, Jianxi Fan, Li Zhang, HS-DCell: A highly scalable DCell-based server-centric topology for data center networks, IEEE/ACM Trans. Netw. (2024) 1–16, http://dx.doi.org/10.1109/TNET.2024.3398628.

[36] Siddhartha Sen, David Shue, Sunghwan Ihm, Michael J Freedman, Scalable, optimal flow routing in datacenters via local link balancing, in: Proceedings of the Ninth ACM Conference on Emerging Networking Experiments and Technologies, 2013, pp. 151–162.

[37] Peng Wang, Hong Xu, Zhixiong Niu, Dongsu Han, Yongqiang Xiong, Expeditus: Congestion-aware load balancing in clos data center networks, in: Proceedings of the Seventh ACM Symposium on Cloud Computing, 2016, pp. 442–455.

[38] Mohammad Al-Fares, Sivasankar Radhakrishnan, Barath Raghavan, Nelson Huang, Amin Vahdat, et al., Hedera: dynamic flow scheduling for data center networks., in: Nsdi, Vol. 10, San Jose, USA, 2010, pp. 89–92.

[39] Andrew R. Curtis, Wonho Kim, Praveen Yalagandula, Mahout: Low-overhead datacenter traffic management using end-host-based elephant detection, in: 2011 Proceedings IEEE INFOCOM, IEEE, 2011, pp. 1629–1637.

[40] Naga Katta, Aditi Ghag, Mukesh Hira, Isaac Keslassy, Aran Bergman, Changhoon Kim, Jennifer Rexford, Clove: Congestion-aware load balancing at the virtual edge, in: Proceedings of the 13th International Conference on Emerging Networking EXperiments and Technologies, 2017, pp. 323–335.

[41] Erico Vanini, Rong Pan, Mohammad Alizadeh, Parvin Taheri, Tom Edsall, Let it flow: Resilient asymmetric load balancing with flowlet switching, in: 14th USENIX Symposium on Networked Systems Design and Implementation, NSDI 17, 2017, pp. 407–420.

[42] Naga Katta, Mukesh Hira, Changhoon Kim, Anirudh Sivaraman, Jennifer Rexford, Hula: Scalable load balancing using programmable data planes, in: Proceedings of the Symposium on SDN Research, 2016, pp. 1–12.

[43] Peng Wang, George Trimponias, Hong Xu, Yanhui Geng, Luopan: Sampling-based load balancing in data center networks, IEEE Trans. Parallel Distrib. Syst. 30 (1) (2018) 133–145.

[44] Yong Liu, Huaxi Gu, Zhaoxing Zhou, Ning Wang, RSLB: Robust and scalable load balancing in software-defined data center networks, IEEE Trans. Netw. Serv. Manag. 19 (4) (2022) 4706–4720.

[45] Advait Dixit, Pawan Prakash, Y Charlie Hu, Ramana Rao Kompella, On the impact of packet spraying in data center networks, in: 2013 Proceedings Ieee Infocom, IEEE, 2013, pp. 2130–2138.

[46] Jiaxin Cao, Rui Xia, Pengkun Yang, Chuanxiong Guo, Guohan Lu, Lihua Yuan, Yixin Zheng, Haitao Wu, Yongqiang Xiong, Dave Maltz, Per-packet load-balanced, low-latency routing for clos-based data center networks, in: Proceedings of the Ninth ACM Conference on Emerging Networking Experiments and Technologies, 2013, pp. 49–60.

[47] David Zats, Tathagata Das, Prashanth Mohan, Dhruba Borthakur, Randy Katz, Detail: Reducing the flow completion time tail in datacenter networks, in: Proceedings of the ACM SIGCOMM 2012 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication, 2012, pp. 139–150.

[48] Mohammad Alizadeh, Albert Greenberg, David A Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, Murari Sridharan, Data center tcp (dctcp), in: Proceedings of the ACM SIGCOMM 2010 Conference, 2010, pp. 63–74.

[49] Balajee Vamanan, Jahangir Hasan, T.N. Vijaykumar, Deadline-aware datacenter tcp (d2tcp), ACM SIGCOMM Comput. Commun. Rev. 42 (4) (2012) 115–126.

[50] Danfeng Shan, Fengyuan Ren, Peng Cheng, Ran Shu, Chuanxiong Guo, Micro-burst in data centers: Observations, analysis, and mitigations, in: 2018 IEEE 26th International Conference on Network Protocols, ICNP, IEEE, 2018, pp. 88–98.

[51] Raj Joshi, Ting Qu, Mun Choon Chan, Ben Leong, Boon Thau Loo, BurstRadar: Practical real-time microburst monitoring for datacenter networks, in: Proceedings of the 9th Asia-Pacific Workshop on Systems, 2018, pp. 1–8.

[52] Zirui Liu, Yikai Zhao, Zhuochen Fan, Tong Yang, Xiaodong Li, Ruwen Zhang, Kaicheng Yang, Zihan Jiang, Zheng Zhong, Yi Huang, et al., Burstbalancer: Do less, better balance for large-scale data center traffic, IEEE Trans. Parallel Distrib. Syst. (2023).

[53] Phillipa Gill, Navendu Jain, Nachiappan Nagappan, Understanding network failures in data centers: measurement, analysis, and implications, in: Proceedings of the ACM SIGCOMM 2011 Conference, 2011, pp. 350–361.

[54] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, Alex C Snoeren, Inside the social network's (datacenter) network, in: Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, 2015, pp. 123–137.

[55] Ka-Cheong Leung, Victor O.K. Li, Daiqin Yang, An overview of packet reordering in transmission control protocol (TCP): problems, solutions, and challenges, IEEE Trans. Parallel Distrib. Syst. 18 (4) (2007) 522–535.

[56] Yilong Geng, Vimalkumar Jeyakumar, Abdul Kabbani, Mohammad Alizadeh, Juggler: a practical reordering resilient network stack for datacenters, in: Proceedings of the Eleventh European Conference on Computer Systems, 2016, pp. 1–16.

[57] Shaojun Zou, Jiawei Huang, Jianxin Wang, Tian He, RMC: Reordering marking and coding for fine-grained load balancing in data centers, IEEE Trans. Commun. 69 (12) (2021) 8363–8374.

[58] Jiawei Huang, Wenjun Lv, Weihe Li, Jianxin Wang, Tian He, QDAPS: Queueing delay aware packet spraying for load balancing in data center, in: 2018 IEEE 26th International Conference on Network Protocols, ICNP, IEEE, 2018, pp. 66–76.

[59] Fayez Alqahtani, Mohammed Amoon, Aida A. Nasr, Reliable scheduling and load balancing for requests in cloud-fog computing, Peer- To- Peer Netw. Appl. 14 (4) (2021) 1905–1916.

[60] V. Gowri, B. Baranidharan, Adaptive probabilistic neural network based edge data center authentication for secure load balancing in fog computing, Appl. Soft Comput. 169 (2025) 112567.

[61] Xiaokun Zheng, Yanqi Zhao, Huilin Li, Ruonan Chen, Dong Zheng, Blockchain-based verifiable privacy-preserving data classification protocol for medical data, Comput. Stand. Interfaces 82 (2022) 103605.

[62] Shreshth Tuli, Nipam Basumatary, Sukhpal Singh Gill, Mohsen Kahani, Rajesh Chand Arya, Gurpreet Singh Wander, Rajkumar Buyya, HealthFog: An ensemble deep learning based smart healthcare system for automatic diagnosis of heart diseases in integrated IoT and fog computing environments, Future Gener. Comput. Syst. 104 (2020) 187–200.

[63] Ji Qi, Xusheng Chen, Yunpeng Jiang, Jianyu Jiang, Tianxiang Shen, Shixiong Zhao, Sen Wang, Gong Zhang, Li Chen, Man Ho Au, et al., Bidl: A high-throughput, low-latency permissioned blockchain framework for datacenter networks, in: Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, 2021, pp. 18–34.

[64] PADSN Wijesekara, Load balancing in blockchain networks: A survey, Int. J. Electr. Electron. Eng. Telecommun. 13 (3) (2024).

[65] Eashan Gupta, Prateesh Goyal, Ilias Marinos, Chenxingyu Zhao, Radhika Mittal, Ranveer Chandra, Dbo: Fairness for cloud-hosted financial exchanges, in: Proceedings of the ACM SIGCOMM 2023 Conference, 2023, pp. 550–563.

[66] Purshotam S. Yadav, Optimizing serverless architectures for ultra-low latency in financial applications, Eur. J. Adv. Eng. Technol. 11 (3) (2024) 146–157.

[67] Suhas Jayaram Subramanya, Daiyaan Arfeen, Shouxu Lin, Aurick Qiao, Zhihao Jia, Gregory R Ganger, Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling, in: Proceedings of the 29th Symposium on Operating Systems Principles, 2023, pp. 642–657.

[68] Sudarsanan Rajasekaran, Manya Ghobadi, Aditya Akella, {CassiNI}:{network-aware} job scheduling in machine learning clusters, in: 21st USENIX Symposium on Networked Systems Design and Implementation, NSDI 24, 2024, pp. 1403–1420.

[69] Daniel E Eisenbud, Cheng Yi, Carlo Contavalli, Cody Smith, Roman Kononov, Eric Mann-Hielscher, Ardas Cilingiroglu, Bin Cheyney, Wentao Shang, Jinnah Dylan Hosein, Maglev: A fast and reliable software network load balancer., in: Nsdi, Vol. 16, 2016, pp. 523–535.

[70] Aaron Archer, Kevin Aydin, Mohammad Hossein Bateni, Vahab Mirrokni, Aaron Schild, Ray Yang, Richard Zhuang, Cache-aware load balancing of data center applications, Proc. VLDB Endow. 12 (6) (2019) 709–723.

[71] Bartek Wydrowski, Robert Kleinberg, Stephen M Rumble, Aaron Archer, Load is not what you should balance: Introducing prequal, in: 21st USENIX Symposium on Networked Systems Design and Implementation, NSDI 24, 2024, pp. 1285–1299.

[72] Mubashir Adnan Qureshi, Yuchung Cheng, Qianwen Yin, Qiaobin Fu, Gautam Kumar, Masoud Moshref, Junhua Yan, Van Jacobson, David Wetherall, Abdul Kabbani, PLB: congestion signals are simple and effective for network load balancing, in: Proceedings of the ACM SIGCOMM 2022 Conference, 2022, pp. 207–218.

[73] Yimin Jiang, Yibo Zhu, Chang Lan, Bairen Yi, Yong Cui, Chuanxiong Guo, A unified architecture for accelerating distributed {dnn} training in heterogeneous {gpu/cpu} clusters, in: 14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 20, 2020, pp. 463–479.

[74] Shubham Chaudhary, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Srinidhi Viswanatha, Balancing efficiency and fairness in heterogeneous GPU clusters for deep learning, in: Proceedings of the Fifteenth European Conference on Computer Systems, 2020, pp. 1–16.

[75] Changji Li, Hongzhi Chen, Shuai Zhang, Yingqian Hu, Chao Chen, Zhenjie Zhang, Meng Li, Xiangchen Li, Dongqing Han, Xiaohui Chen, et al., ByteGraph: a high-performance distributed graph database in ByteDance, Proc. VLDB Endow. 15 (12) (2022) 3306–3318.

[76] NGINX, 2004, (Accessed 19 February 2024). https://nginx.org/.

[77] Linux Virtual Server (LVS), 1998, (Accessed 19 February 2024). http://www.linuxvirtualserver.org/.

[78] HAProxy, 2000, (Accessed 19 February 2024). http://www.haproxy.org/.

[79] Rohan Gandhi, Y. Charlie Hu, Ming Zhang, Yoda: A highly available layer-7 load balancer, in: Proceedings of the Eleventh European Conference on Computer Systems, 2016, pp. 1–16.

[80] Reuven Cohen, Matty Kadosh, Alan Lo, Qasem Sayah, LB scalability: Achieving the right balance between being stateful and stateless, IEEE/ACM Trans. Netw. 30 (1) (2021) 382–393.

[81] Mohammad Hosseini, Sina Darabi, Amir Hossein Jahangir, Ali Movaghar, Yuz: Improving performance of cluster-based services by near-l4 session-persistent load balancing, IEEE Trans. Netw. Serv. Manag. (2023).

[82] Yazhi Liu, Jiye Zhang, Wei Li, Qianqian Wu, Pengmiao Li, Load balancing oriented predictive routing algorithm for data center networks, Futur. Internet 13 (2) (2021) 54.

[83] Binjie He, Dong Zhang, Chang Zhao, Hidden Markov model-based load balancing in data center networks, Comput. J. 63 (10) (2020) 1449–1462.

[84] Brian Chang, Aditya Akella, Loris D'Antoni, Kausik Subramanian, Learned load balancing, in: Proceedings of the 24th International Conference on Distributed Computing and Networking, 2023, pp. 177–187.

[85] Alex M.R. Ruelas, Christian Esteve Rothenberg, A load balancing method based on artificial neural networks for knowledge-defined data center networking, in: Proceedings of the 10th Latin America Networking Conference, 2018, pp. 106–109.

[86] Yuan Liu, Wenxin Li, Wenyu Qu, Heng Qi, BULB: lightweight and automated load balancing for fast datacenter networks, in: Proceedings of the 51st International Conference on Parallel Processing, 2022, pp. 1–11.

[87] Bilal Babayigit, Banu Ulu, Deep learning for load balancing of SDN-based data center networks, Int. J. Commun. Syst. 34 (7) (2021) e4760.

[88] Negar Rikhtegar, Omid Bushehrian, Manijeh Keshtgari, DeepRLB: A deep reinforcement learning-based load balancing in data center networks, Int. J. Commun. Syst. 34 (15) (2021) e4912.

[89] Vasileios Tosounidis, Georgios Pavlidis, Ilias Sakellariou, Deep Q-learning for load balancing traffic in SDN networks, in: 11th Hellenic Conference on Artificial Intelligence, 2020, pp. 135–143.

[90] Zeng Xiangyun, Wu Lijun, Li Zhiyuan, Jing Yulin, Deep reinforcement learning with graph convolutional networks for load balancing in SDN-based data center networks, in: 2021 18th International Computer Conference on Wavelet Active Media Technology and Information Processing, ICCWAMTIP, IEEE, 2021, pp. 344–352.

[91] Jinbin Hu, Wangqing Luo, Yi He, Jing Wang, Dengyong Zhang, Deep reinforcement learning based load balancing for heterogeneous traffic in datacenter networks, in: International Conference on Algorithms and Architectures for Parallel Processing, Springer, 2023, pp. 270–289.

[92] Guoyong Jiang, Wenting Wei, Kun Wang, Chengding Pang, Yong Liu, DRL-TAL: Deep reinforcement learning-based traffic-aware load balancing in data center networks, in: GLOBECOM 2023-2023 IEEE Global Communications Conference, IEEE, 2023, pp. 928–933.

[93] C. Fancy, M. Pushpalatha, Intelligence-enabled approach for load balancing in software-defined data center networks, Int. J. Commun. Syst. 34 (9) (2021).

[94] Feifan Hao, Shan Jing, Chuan Zhao, Link load balancing scheme for elephant flow in SDN data center, in: 2023 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking, ISPA/BDCloud/SocialCom/SustainCom, IEEE, 2023, pp. 1026–1033.

[95] Qinliang Lin, Zhibo Gong, Qiaoling Wang, Jinlong Li, Rilnet: A reinforcement learning based load balancing approach for datacenter networks, in: Machine Learning for Networking: First International Conference, MLN 2018, Paris, France, November 27–29, 2018, Revised Selected Papers 1, Springer, 2019, pp. 44–55.

[96] Jiyoon Lim, Jae-Hyoung Yoo, James Won-Ki Hong, Reinforcement learning based load balancing for data center networks, in: 2021 IEEE 7th International Conference on Network Softwarization, NetSoft, IEEE, 2021, pp. 151–155.

[97] Wai-Xi Liu, Jinjie Lu, Jun Cai, Yinghao Zhu, Sen Ling, Qingchun Chen, DRL-PLink: Deep reinforcement learning with private link approach for mix-flow scheduling in software-defined data-center networks, IEEE Trans. Netw. Serv. Manag. 19 (2) (2021) 1049–1064.

[98] Zehua Guo, Yang Xu, Ya-Feng Liu, Sen Liu, H Jonathan Chao, Zhi-Li Zhang, Yuanqing Xia, AggreFlow: Achieving power efficiency, load balancing, and quality of service in data center networks, IEEE/ACM Trans. Netw. 29 (1) (2020) 17–33.

[99] Negar Rikhtegar, Manijeh Keshtgari, Omid Bushehrian, Guy Pujolle, BiTE: a dynamic bi-level traffic engineering model for load balancing and energy efficiency in data center networks, Appl. Intell. 51 (2021) 4623–4648.

[100] Yihao He, Zebin Lu, Junru Lei, Shuhua Deng, Xieping Gao, Joint optimization of energy saving and load balancing for data center networks based on software defined networks, Concurr. Comput.: Pr. Exp. 33 (9) (2021) e6134.

[101] Md Naimul Pathan, Maisha Muntaha, Selina Sharmin, Sajeeb Saha, Md Ashraf Uddin, Fernaz Narin Nur, Sunil Aryal, Priority based energy and load aware routing algorithms for SDN enabled data center network, Comput. Netw. 240 (2024) 110166.

[102] Syed Mohammad Irteza, Hafiz Mohsin Bashir, Talal Anwar, Ihsan Ayyub Qazi, Fahad Rafique Dogar, Efficient load balancing over asymmetric datacenter topologies, Comput. Commun. 127 (2018) 1–12.

[103] Yongan Shu, Dongsong Gao, A dynamic multipath load balancing algorithm based on particle swarm optimization in DCN, in: 2023 IEEE 14th International Conference on Software Engineering and Service Science, ICSESS, IEEE, 2023, pp. 126–130.

[104] Yao Guohao, Wu Muqing, Yang Xu, A data center load balancing algorithm based on artificial bee colony algorithm, in: 2020 IEEE 6th International Conference on Computer and Communications, ICCC, IEEE, 2020, pp. 1770–1775.

[105] Jiaqi Zheng, Qiming Zheng, Xiaofeng Gao, Guihai Chen, Dynamic load balancing in hybrid switching data center networks with converters, in: Proceedings of the 48th International Conference on Parallel Processing, ICPP '19, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450362955, 2019, URL https://doi.org/10.1145/3337821.3337898.

[106] Wei-Kang Chung, Yun Li, Chih-Heng Ke, Sun-Yuan Hsieh, Albert Y Zomaya, Rajkumar Buyya, Dynamic parallel flow algorithms with centralized scheduling for load balancing in cloud data center networks, IEEE Trans. Cloud Comput. 11 (1) (2021) 1050–1064.

[107] Aymen Hasan Alawadi, Sándor Molnár, Oddlab: fault-tolerant aware load-balancing framework for data center networks, Ann. Telecommun. (2022) 1–22.

[108] Junbo Zhang, Yu Zheng, Dekang Qi, Deep spatio-temporal residual networks for citywide crowd flows prediction, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 31, 2017.

[109] Guoqing Liu, Xingxuan Wang, A modified round-robin load balancing algorithm based on content of request, in: 2018 5th International Conference on Information Science and Control Engineering, ICISCE, IEEE, 2018, pp. 66–72.

[110] Yang Wang, Yutong Li, Ting Wang, Gang Liu, Towards an energy-efficient data center network based on deep reinforcement learning, Comput. Netw. 210 (2022) 108939.

[111] Lawrence Rabiner, Biinghwang Juang, An introduction to hidden Markov models, Ieee Assp Mag. 3 (1) (1986) 4–16.

[112] Albert Mestres, Alberto Rodriguez-Natal, Josep Carner, Pere Barlet-Ros, Eduard Alarcón, Marc Solé, Victor Muntés-Mulero, David Meyer, Sharon Barkai, Mike J Hibbett, et al., Knowledge-defined networking, ACM SIGCOMM Comput. Commun. Rev. 47 (3) (2017) 2–10.

[113] Rogério Leão Santos De Oliveira, Christiane Marie Schweitzer, Ailton Akira Shinoda, Ligia Rodrigues Prete, Using mininet for emulation and prototyping software-defined networks, in: 2014 IEEE Colombian Conference on Communications and Computing, COLCOM, IEEE, 2014, pp. 1–6.

[114] Jan Medved, Robert Varga, Anton Tkacik, Ken Gray, Opendaylight: Towards a model-driven sdn controller architecture, in: Proceeding of IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks 2014, IEEE, 2014, pp. 1–6.

[115] Kurt S. Wilcox, Wayne Guillien, Frank Tsao, Floodlight, 2012, US Patent App. 29/397, 632.

[116] Christopher J.C.H. Watkins, Peter Dayan, Q-learning, Mach. Learn. 8 (1992) 279–292.

[117] Volodymyr Mnih, Playing atari with deep reinforcement learning, 2013, arXiv preprint arXiv:1312.5602.

[118] Diederik P. Kingma, Adam: A method for stochastic optimization, 2014, arXiv preprint arXiv:1412.6980.

[119] Thomas N. Kipf, Max Welling, Semi-supervised classification with graph convolutional networks, 2016, arXiv preprint arXiv:1609.02907.

[120] Gabriel Barth-Maron, Matthew W Hoffman, David Budden, Will Dabney, Dan Horgan, Dhruva Tb, Alistair Muldal, Nicolas Heess, Timothy Lillicrap, Distributed distributional deterministic policy gradients, 2018, arXiv preprint arXiv:1804.08617.

[121] Wei Wang, Yi Sun, Kai Zheng, Mohamed Ali Kaafar, Dan Li, Zhongcheng Li, Freeway: Adaptively isolating the elephant and mice flows on different transmission paths, in: 2014 IEEE 22nd International Conference on Network Protocols, IEEE, 2014, pp. 362–367.

[122] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov, Proximal policy optimization algorithms, 2017, arXiv preprint arXiv:1707.06347.

[123] Jixin Zhao, Shukui Zhang, Yang Zhang, Li Zhang, Hao Long, Deep reinforcement learning-based routing optimization algorithm for edge data center, in: 2021 IEEE Symposium on Computers and Communications, ISCC, IEEE, 2021, pp. 1–7.

[124] Milan Patel, Brian Naughton, Caroline Chan, Nurit Sprecher, Sadayuki Abeta, Adrian Neal, et al., Mobile-edge computing introductory technical white paper, White Pap. Mobile- Edge Comput. (MEC) Ind. Initiat. 29 (2014) 854–864.

[125] T.P. Lillicrap, Continuous control with deep reinforcement learning, 2015, arXiv preprint arXiv:1509.02971.

[126] IEA, Digitalisation and Energy, Technical report, International Energy Agency, Paris, 2017, URL https://www.iea.org/reports/digitalisation-and-energy. Licence: CC BY 4.0.

[127] Eric Masanet, Arman Shehabi, Nuoa Lei, Sarah Smith, Jonathan Koomey, Recalibrating global data center energy-use estimates, Science 367 (6481) (2020) 984–986.

[128] Sustainable innovation & technology - google sustainability, 2023, URL https://sustainability.google.

[129] Climate solutions - amazon sustainability, 2023, URL https://sustainability.aboutamazon.com/climate-solutions.

[130] Christodoulos A. Floudas, Xiaoxia Lin, Mixed integer linear programming in process scheduling: Modeling, algorithms, and applications, Ann. Oper. Res. 139 (2005) 131–162.

[131] Shengxu Xie, Guyu Hu, Changyou Xing, Yaqun Liu, Online elephant flow prediction for load balancing in programmable switch based DCN, IEEE Trans. Netw. Serv. Manag. (2023).

[132] Weifeng Zhang, Dongfang Ling, Yuanrong Zhang, Pengfei Li, Guo Chen, Achieving optimal edge-based congestion-aware load balancing in data center networks, in: 2020 IFIP Networking Conference, Networking, IEEE, 2020, pp. 109–117.
[133] Sultan Alanazi, Bechir Hamdaoui, CAFT: Congestion-aware fault-tolerant load balancing for three-tier clos data centers, in: 2020 International Wireless Communications and Mobile Computing, IWCMC, IEEE, 2020, pp. 1746–1751.
[134] Chung-Ming Cheung, Ka-Cheong Leung, DFFR: a flow-based approach for distributed load balancing in data center networks, Comput. Commun. 116 (2018) 1–8.
[135] Yu Xia, Jinsong Wu, Jingwen Xia, Ting Wang, Sun Mao, Multipath-aware TCP for data center traffic load-balancing, in: 2021 IEEE/ACM 29th International Symposium on Quality of Service, IWQOS, IEEE, 2021, pp. 1–6.
[136] Hasnain Ahmed, Muhammad Junaid Arshad, Shah Muhammad, Sarfraz Ahmad, Amjad Hussain Zahid, Queue length-based load balancing in data center networks, Int. J. Commun. Syst. 33 (14) (2020) e4472.
[137] Lu Liu, Yong Jiang, Gengbiao Shen, Qing Li, Dong Lin, Li Li, Yi Wang, An SDN-based hybrid strategy for load balancing in data center networks, in: 2019 IEEE Symposium on Computers and Communications, ISCC, IEEE, 2019, pp. 1–6.
[138] Tao Zhang, Ran Huang, Yuanzhen Hu, Yangfan Li, Shaojun Zou, Qianqiang Zhang, Xin Liu, Chang Ruan, Load balancing with deadline-driven parallel data transmission in data center networks, IEEE Internet Things J. 10 (2) (2022) 1171–1191.
[139] Fu Wang, Haipeng Yao, Qi Zhang, Jingjing Wang, Ran Gao, Dong Guo, Mohsen Guizani, Dynamic distributed multi-path aided load balancing for optical data center networks, IEEE Trans. Netw. Serv. Manag. 19 (2) (2021) 991–1005.
[140] Sheeba Memon, Jiawei Huang, Hussain Saajid, Naadiya Khuda Bux, Arshad Saleem, Yazan Aljeroudi, Novel multi-level dynamic traffic load-balancing protocol for data center, Symmetry 11 (2) (2019) 145.
[141] Weimin Gao, Jiawei Huang, Ning Jiang, Zhaoyi Li, Shaojun Zou, Zhidong He, Jianxin Wang, HPLB: High precision load balancing based on in-band network telemetry in data center networks, Peer- To- Peer Netw. Appl. 15 (6) (2022) 2503–2515.
[142] Shaojun Zou, Jiawei Huang, Wanchun Jiang, Jianxin Wang, Achieving high utilization of flowlet-based load balancing in data center networks, Future Gener. Comput. Syst. 108 (2020) 546–559.
[143] Xinglong Diao, Huaxi Gu, Xiaoshan Yu, Liang Qin, Changyun Luo, Flex: A flowlet-level load balancing based on load-adaptive timeout in DCN, Future Gener. Comput. Syst. 130 (2022) 219–230.
[144] Fujie Fan, Bing Hu, Kwan L. Yeung, Routing in black box: Modularized load balancing for multipath data center networks, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, IEEE, 2019, pp. 1639–1647.
[145] Wai-xi Liu, Jun Cai, Sen Ling, Jian-Yu Zhang, Qingchun Chen, QALL: Distributed queue-behavior-aware load balancing using programmable data planes, IEEE Trans. Netw. Serv. Manag. (2023).
[146] Jingling Liu, Jiawei Huang, Wenjun Lv, Jianxin Wang, APS: Adaptive packet spraying to isolate mix-flows in data center network, IEEE Trans. Cloud Comput. 10 (2) (2020) 1038–1051.
[147] Wai-Xi Liu, Jun Cai, Ying-Hao Zhu, Jun-Ming Luo, Jin Li, Load balancing inside programmable data planes based on network modeling prediction using a GNN with network behaviors, Comput. Netw. 227 (2023) 109695.
[148] Yifei Lu, Zhengzhi Xu, Xu Ma, FAMG: A flow-aware and mixed granularity method for load-balancing in data center networks, Comput. Commun. 209 (2023) 415–428.
[149] Liang Qin, Wenting Wei, Xinglong Diao, FlowDecider: End-host driven proactive load balancing for data center networks, in: 2021 IEEE 21st International Conference on Communication Technology, ICCT, IEEE, 2021, pp. 931–936.
[150] Qingyu Shi, Fang Wang, Dan Feng, IntFlow: Integrating per-packet and per-flowlet switching strategy for load balancing in datacenter networks, IEEE Trans. Netw. Serv. Manag. 17 (3) (2020) 1377–1388.
[151] Zhengzhi Xu, Yifei Lu, Xu Ma, Bobble: A mixed routing-granularity distributed load balancing for data center networks, in: 2021 IEEE 23rd Int Conf on High Performance Computing & Communications; 7th Int Conf on Data Science & Systems; 19th Int Conf on Smart City; 7th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application, HPCC/DSS/SmartCity/DependSys, IEEE, 2021, pp. 470–477.
[152] Tao Zhang, Qianqiang Zhang, Yasi Lei, Shaojun Zou, Juan Huang, Fangmin Li, Load balancing with traffic isolation in data center networks, Future Gener. Comput. Syst. 127 (2022) 126–141.
[153] Jingling Liu, Jiawei Huang, Weihe Li, Jianxin Wang, AG: Adaptive switching granularity for load balancing with asymmetric topology in data center network, in: 2019 IEEE 27th International Conference on Network Protocols, ICNP, IEEE, 2019, pp. 1–11.
[154] Jinbin Hu, Jiawei Huang, Wenjun Lyu, Weihe Li, Zhaoyi Li, Wenchao Jiang, Jianxin Wang, Tian He, Adjusting switching granularity of load balancing for heterogeneous datacenter traffic, IEEE/ACM Trans. Netw. 29 (5) (2021) 2367–2384.
[155] Weimin Gao, Jiawei Huang, Zhaoyi Li, Shaojun Zou, Jianxin Wang, PDLB: Path diversity-aware load balancing with adaptive granularity in data center networks, J. Cloud Comput. 12 (1) (2023) 173.
[156] Tao Zhang, Shengli He, Xiao Zeng, Xin Wu, Ku Jin, Yuanzhen Hu, Chang Ruan, Shaojun Zou, Jinbin Hu, Fangmin Li, HG: Leveraging hybrid switching granularity to balance heterogeneous data center traffic load for cloud-based industrial applications, IEEE Trans. Ind. Inform. (2024).
[157] Cha Hwan Song, Xin Zhe Khooi, Raj Joshi, Inho Choi, Jialin Li, Mun Choon Chan, Network load balancing with in-network reordering support for rdma, in: Proceedings of the ACM SIGCOMM 2023 Conference, 2023, pp. 816–831.
[158] Jin Wang, Shuying Rao, Ying Liu, Pradip Kumar Sharma, Jinbin Hu, Load balancing for heterogeneous traffic in datacenter networks, J. Netw. Comput. Appl. 217 (2023) 103692.
[159] Chuanxiong Guo, Haitao Wu, Zhong Deng, Gaurav Soni, Jianxi Ye, Jitu Padhye, Marina Lipshteyn, RDMA over commodity ethernet at scale, in: Proceedings of the 2016 ACM SIGCOMM Conference, 2016, pp. 202–215.
[160] Changgang Zheng, Benjamin Rienecker, Noa Zilberman, QCMP: Load Balancing via In-Network Reinforcement Learning, in: Proceedings of the 2nd ACM SIGCOMM Workshop on Future of Internet Routing & Addressing, 2023, pp. 35–40.
[161] Menghao Zhang, Guanyu Li, Cheng Guo, Renyu Yang, Shicheng Wang, Han Bao, Xiao Li, Mingwei Xu, Tianyu Wo, Chunming Hu, SuperFE: A scalable and flexible feature extractor for ML-based traffic analysis applications, 2025.
[162] Federico Parola, Roberto Procopio, Roberto Querio, Fulvio Risso, Comparing user space and in-kernel packet processing for edge data centers, ACM SIGCOMM Comput. Commun. Rev. 53 (1) (2023) 14–29.
[163] Patric LR Izolan, Ivan M Lopes Júnior, Ester S Oribes, Braulio M de Souza, Angelo V Crestani, Paulo S de Souza, Marcelo C Luizelli, Fábio D Rossi, User-level network programmability: A scalability study for data center infrastructure, in: 2024 International Symposium on Computer Architecture and High Performance Computing Workshops, SBAC-PADW, IEEE, 2024, pp. 42–49.
[164] Ali Abir Shuvro, Mohammad Shian Khan, Monzur Rahman, Faisal Hussain, Md Moniruzzaman, Md Sakhawat Hossen, Transformer based traffic flow forecasting in SDN-VANET, IEEE Access 11 (2023) 41816–41826.
[165] Junjie Guo, Chungang Yang, Fuqiang Li, Ru Dong, Yanbo Song, Shiwen Kou, IDEAL: Intent driven emerging anti-congestion router with load-balance for SDN, in: 2023 6th World Conference on Computing and Communication Technologies, WCCCT, IEEE, 2023, pp. 39–45.
[166] Adeel Rafiq, Muhammad Afaq, Wang-Cheol Song, Intent-based networking with proactive load distribution in data center using IBN manager and smart path manager, J. Ambient. Intell. Humaniz. Comput. 11 (11) (2020) 4855–4872.